

# **Smart EV Charging Slot Reservation with Dynamic Slot Suggestion**

by

**Asude Beyza Demirboğa**

**Engineering Project Report**

**Yeditepe University**

**Faculty of Engineering**

**Department of Computer Engineering**

**2025**



## **Smart EV Charging Slot Reservation with Dynamic Slot Suggestion**

APPROVED BY:

|                                               |       |
|-----------------------------------------------|-------|
| Assist. Prof. Dr. Tacha Serif<br>(Supervisor) | ..... |
| Assist. Prof. Dr. Mustafa Bülent Mutluoğlu    | ..... |
| Academic Staff Gülşah Gökhan Gökçek           | ..... |

DATE OF APPROVAL: 18/06/2025

## ACKNOWLEDGEMENTS

First of all, I would like to express my deepest gratitude to my advisor, Assist. Prof. Dr. Tacha SERİF, for his invaluable guidance and continuous support throughout the development of this project.

I would also like to sincerely thank my family for their unwavering support during this journey. I am especially grateful to my dear mother, who stood by me despite her worries and always tried to be my biggest supporter; and to my father, who reminded me that nothing is truly impossible—just time-consuming. A heartfelt thank you goes to my sister Nurneda, who has always been the one to make me laugh the most, especially during this process, and to my little brother İhsan, who helped by simply leaving me alone when I needed space.

I am deeply thankful to my uncle, who has always been my role model. Growing up in admiration of him, I feel proud to pursue the same profession, and sometimes still can't believe I do. I thank him for being the amazing uncle who never hesitates to debug all my code from start to finish and always comes to my aid whenever I face a challenge.

Lastly, I want to express my appreciation to three wonderful friends who made this journey even more meaningful. To Ece, for sharing the same stress before exams and the excitement of school days; to Müberra, for bringing stress management and spontaneous flight searches into my life; and to my dear friend Eren, for his calming recitals that helped ease my anxiety. Studying with you was truly a privilege.

## **ABSTRACT**

### **Smart EV Charging Slot Reservation with Dynamic Slot Suggestion**

As electric vehicle (EV) adoption accelerates, the efficient management of public charging infrastructure has become an increasingly critical challenge, particularly in regions such as Turkey where the number of registered EVs has surged dramatically. Despite rapid growth in charging infrastructure, existing first-come, first-served models often lead to congestion, inefficient charger utilization, and extended waiting times, which degrade the user experience and hinder the scalability of electric mobility. Furthermore, current reservation systems typically lack predictive capabilities, providing only real-time availability without accounting for future demand or dynamic user behavior.

To address these limitations, this project proposes an intelligent EV charging slot recommendation and reservation system. The system leverages nearly real-time factors—including vehicle battery level, estimated time of arrival (ETA), station availability predictions, and active reservations to compute a Slot Fit Score for each station and time slot. This enables personalized and optimized charging slot recommendations for users. The solution integrates a predictive machine learning model and a user-friendly iOS mobile application to enhance both user satisfaction and overall charger utilization. The system was developed and evaluated using the Codalab Smart Mobility Data Challenge dataset [1], achieving strong predictive performance and demonstrating its potential to improve the efficiency and scalability of modern EV charging networks.

## ÖZET

### Elektrikli Araçlar için Dinamik Slot Önerili Akıllı Rezervasyon Sistemi

Elektrikli araçların benimsenmesinin hızlanmasıyla birlikte, kamuya açık şarj altyapısının etkin yönetimi giderek daha kritik bir sorun haline gelmiştir. Özellikle Türkiye gibi elektrikli araç sayısının hızla arttığı bölgelerde bu ihtiyaç daha da belirgindir. Şarj altyapısındaki hızlı büyümeye rağmen, mevcut "ilk gelen kullanır" modeli sıklıkla istasyonlarda yoğunluk, verimsiz şarj noktası kullanımı ve uzun bekleme sürelerine yol açmakta; bu da kullanıcı deneyimini olumsuz etkileyerek elektrikli mobilitenin ölçeklenebilirliğini sınırlamaktadır. Ayrıca mevcut rezervasyon sistemleri genellikle gelecekteki talebi veya dinamik kullanıcı davranışlarını dikkate almadan yalnızca anlık kullanılabilirlik bilgisi sunmaktadır.

Bu eksiklikleri gidermek amacıyla, bu proje akıllı bir Elektrikli Araç Şarj Slotu Öneri ve Rezervasyon sistemi geliştirmektedir. Sistem, araç batarya seviyesi, tahmini varış zamanı, istasyon kullanılabilirlik tahminleri ve mevcut rezervasyonlar gibi neredeyse gerçek zamanlı faktörleri değerlendirerek her istasyon ve zaman dilimi için bir Slot Fit Skoru hesaplamaktadır. Bu sayede kullanıcılara kişiselleştirilmiş ve optimize edilmiş şarj slotu önerileri sunulmaktadır. Çözüm, tahmine dayalı bir makine öğrenimi modeli ile kullanıcı dostu bir iOS mobil uygulamasını entegre ederek hem kullanıcı memnuniyetini hem de şarj altyapısının genel verimliliğini arttırmayı hedeflemektedir. Sistem, CodaLab Smart Mobility Data Challenge veri seti [1] kullanılarak geliştirilmiş ve değerlendirilmiş, güçlü tahmin performansı sergileyerek modern elektrikli araç şarj ağlarının verimliliği ve ölçeklenebilirliğini artırma potansiyelini ortaya koymuştur.

# TABLE OF CONTENTS

## Contents

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| <b>1</b> | <b>INTRODUCTION</b>                                              | <b>1</b>  |
| <b>2</b> | <b>BACKGROUND</b>                                                | <b>3</b>  |
| 2.1      | EV Charging Management and Reservation Systems . . . . .         | 3         |
| 2.2      | Security and Privacy Based Reservation Systems . . . . .         | 9         |
| 2.3      | Slot Recommendation and Scheduling Systems . . . . .             | 10        |
| 2.4      | EV Charging Infrastructure Focus and Grid Optimization . . . . . | 12        |
| <b>3</b> | <b>METHODOLOGIES</b>                                             | <b>14</b> |
| 3.1      | Regression Algorithms . . . . .                                  | 14        |
| 3.1.1    | Decision Tree Regression . . . . .                               | 16        |
| 3.1.2    | Random Forest Regression . . . . .                               | 17        |
| 3.1.3    | Gradient Boosting Regression . . . . .                           | 18        |
| 3.2      | Software & Tools . . . . .                                       | 19        |
| 3.2.1    | IOS Native . . . . .                                             | 19        |
| 3.2.2    | Google APIs & Firebase Console . . . . .                         | 20        |
| 3.2.3    | Google Colab . . . . .                                           | 21        |
| 3.3      | Datasets . . . . .                                               | 23        |
| 3.3.1    | Paris Charging Station Dataset . . . . .                         | 23        |
| 3.3.2    | ST-EVCDP Dataset . . . . .                                       | 24        |
| 3.4      | Test Methodologies . . . . .                                     | 24        |
| 3.4.1    | Evaluation Metrics . . . . .                                     | 25        |
| 3.4.2    | SUS Test . . . . .                                               | 27        |
| <b>4</b> | <b>ANALYSIS &amp; DESIGN</b>                                     | <b>28</b> |
| 4.1      | Functional Requirement . . . . .                                 | 28        |
| 4.2      | Non-Functional Requirements . . . . .                            | 30        |
| 4.3      | Sequence Diagrams . . . . .                                      | 32        |
| 4.3.1    | Recommended Stations Sequence Diagram . . . . .                  | 32        |

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| 4.3.2    | Reservation Sequence Diagram . . . . .          | 34        |
| <b>5</b> | <b>IMPLEMENTATION</b>                           | <b>35</b> |
| 5.1      | Availability Prediction Model . . . . .         | 35        |
| 5.1.1    | Dataset . . . . .                               | 35        |
| 5.1.2    | Feature Extraction and Model Training . . . . . | 36        |
| 5.2      | Mobile Application & GUI . . . . .              | 38        |
| 5.2.1    | Swift & SwiftUI Library . . . . .               | 40        |
| 5.3      | API & Database . . . . .                        | 41        |
| <b>6</b> | <b>TEST &amp; RESULTS</b>                       | <b>44</b> |
| 6.1      | Model Evaluation . . . . .                      | 44        |
| 6.2      | Application Reservation Test . . . . .          | 45        |
| 6.3      | Application Usability Tests . . . . .           | 48        |
| 6.3.1    | SUS Test . . . . .                              | 49        |
| 6.3.2    | Application Questionnaire . . . . .             | 49        |
| <b>7</b> | <b>CONCLUSION</b>                               | <b>51</b> |
| <b>8</b> | <b>Bibliography</b>                             | <b>52</b> |

## LIST OF FIGURES

### List of Figures

|      |                                                                                                               |    |
|------|---------------------------------------------------------------------------------------------------------------|----|
| 2.1  | SecCharge user interface displaying nearby reservable EV charging stations on a map view. [4]                 | 3  |
| 2.2  | Illustrates the architecture of the S2RC system. [5]                                                          | 4  |
| 2.3  | Overview of proposed EV charging management. [6]                                                              | 5  |
| 2.4  | Images from the smartphone app of the system. [7]                                                             | 5  |
| 2.5  | the system model for interoperable EV charging reservations based on the EMSA framework. [8]                  | 6  |
| 2.6  | Illustrates the cloud-based EV charging architecture. [9]                                                     | 7  |
| 2.7  | Illustrates the Smart EVC platform architecture. [10]                                                         | 7  |
| 2.8  | Illustrates the cloud-based EV charging architecture. [12]                                                    | 8  |
| 2.9  | The reservation-based electric vehicle charging service mode. [13]                                            | 9  |
| 2.10 | illustrates the privacy-preserving reservation protocol, where EV users securely reserve charging slots. [14] | 10 |
| 2.11 | EV charging station management solution overview. [21]                                                        | 12 |
| 3.1  | Visualization of linear regression [25].                                                                      | 14 |
| 3.2  | A visual representation of a Non-Linear Regression [26].                                                      | 15 |
| 3.3  | Decision Tree Regression [27].                                                                                | 16 |
| 3.4  | Random Forest Regression [28].                                                                                | 17 |
| 3.5  | Gradient Boosting Regression [29].                                                                            | 18 |
| 3.6  | Xcode Environment.                                                                                            | 20 |
| 3.7  | Firebase Console.                                                                                             | 21 |
| 3.8  | Colab Environment.                                                                                            | 22 |
| 3.9  | Distribution of charging stations. [1]                                                                        | 23 |
| 3.10 | Geographical distribution of EV charging piles in Shenzhen. [30]                                              | 24 |
| 4.1  | Recommended Stations Sequence Diagram                                                                         | 32 |
| 4.2  | Rezervation Process Sequence Diagram                                                                          | 34 |
| 5.1  | Overall Architecture of the System                                                                            | 35 |
| 5.2  | First five row from Dataset                                                                                   | 36 |

|     |                                                       |    |
|-----|-------------------------------------------------------|----|
| 5.3 | Explore tab screens. . . . .                          | 39 |
| 5.4 | Car models collection . . . . .                       | 42 |
| 5.5 | Stations collection . . . . .                         | 43 |
| 5.6 | Stations status sub-collection . . . . .              | 43 |
| 6.1 | Reservation Process . . . . .                         | 46 |
| 6.2 | Reservations and Recommended Station Update . . . . . | 47 |
| 6.3 | Reservation Limit . . . . .                           | 48 |
| 6.4 | Usability questions' response average. . . . .        | 50 |

## LIST OF TABLES

### List of Tables

|   |                                                            |    |
|---|------------------------------------------------------------|----|
| 1 | System Usability Scale (SUS) Questionnaire Items . . . . . | 27 |
| 2 | Regression Model Evaluation Results . . . . .              | 44 |
| 3 | Usability Test Questions . . . . .                         | 50 |

## **LIST OF SYMBOLS / ABBREVIATIONS**

|                      |                                                      |
|----------------------|------------------------------------------------------|
| <b>EV</b>            | Electric Vehicle                                     |
| <b>ETA</b>           | Estimated Time of Arrival                            |
| <b>OCPP</b>          | Open Charge Point Protocol                           |
| <b>RMSE</b>          | Root Mean Squared Error                              |
| <b>MAE</b>           | Mean Absolute Error                                  |
| <b>R<sup>2</sup></b> | Coefficient of Determination                         |
| <b>SUS</b>           | System Usability Scale                               |
| <b>MVVM</b>          | Model-View-ViewModel (Software Architecture Pattern) |
| <b>OBD2</b>          | On-Board Diagnostics II                              |

# 1 INTRODUCTION

As electric vehicles (EVs) continue to grow in popularity, driven by both environmental concerns and supportive governmental policies, efficient management of public charging infrastructure has become an increasingly urgent challenge. In Turkey, the number of registered EVs surged dramatically, increasing by over 130% within a single year to reach 183,776 vehicles in 2024 [2]. Although charging infrastructure expanded considerably—from 11,812 to 26,046 stations [3]—this growth remains insufficient to match the rapidly increasing demand. The prolonged charging times typical of EVs mean that popular charging stations often experience severe congestion, inefficient utilization of resources, and extended waiting times, especially during peak usage periods such as holidays and rush hours. Without a well-optimized system for managing charging sessions, these challenges will likely escalate, significantly affecting user satisfaction and potentially hindering the broader adoption and scalability of electric mobility solutions in Turkey.

Current EV charging stations typically operate on a first-come, first-served model. Even when reservation options exist, most systems only indicate the current availability at the time of booking without forecasting future demand. Such limitations frequently lead to user dissatisfaction when users arrive to find their reserved charging station unexpectedly crowded. Moreover, the coexistence of reserved and non-reserved usage patterns without proper guidance or dynamic scheduling exacerbates inefficient charger utilization and creates inconsistent user experiences.

To effectively address these challenges, this project introduces an intelligent EV charging slot recommendation and reservation system. This system dynamically suggests the most suitable charging slots for drivers based on nearly real-time data inputs, including the vehicle's battery level, estimated time of arrival (ETA), predicted station availability, and current reservations. By incorporating advanced predictive capabilities, the proposed system significantly improves upon existing reservation models. Instead of requiring manual selection by users, it calculates a comprehensive "Slot Fit Score" for each available station and time slot combination. This scoring approach integrates machine learning-based predictions to rank charging options based on projected availability and precise ETA computations. Consequently, users receive personalized recommendations, allowing them to effortlessly reserve preferred charging slots through an intuitive mobile application.

The technological integration behind this solution leverages powerful tools such as Firebase for real-time data synchronization and secure user authentication, Google APIs for accurate navigation and ETA estimations, and sophisticated machine learning algorithms for predictive modeling of station availability. These combined technologies ensure the system remains responsive, accurate, and user-friendly, thereby substantially enhancing the overall EV charg-

ing experience and helping to manage the growing demand more effectively. Accordingly, the report is structured as follows:

- **Section 2: Background** This section reviews existing literature on EV charging management, summarizing state-of-the-art approaches for infrastructure management, reservation systems, slot recommendation methods, and solutions addressing security, privacy, and grid optimization.
- **Section 3: Methodologies** This section offers a concise overview of the machine learning models used in the system's predictive component. It also highlights key development technologies and tools utilized throughout the project, such as Swift for iOS application development and Firebase for backend management. Additionally, the section discusses datasets relevant to the project's development. It concludes by detailing the evaluation methods employed, including predictive model performance metrics and usability testing.
- **Section 4: Analysis & Design** clearly defines the functional and non-functional requirements established for the system, providing a detailed framework for expected system behaviors, quality attributes, and operational constraints. Sequence diagrams are included to visually illustrate the critical interactions and data flows involved in both the recommended station generation and the reservation processes, offering insights into the internal logic and system integration.
- **Section 5: Implementation** explains the practical realization of the system components, including the development and integration of the availability prediction model. It provides specifics about the mobile application architecture built using SwiftUI, the integration with APIs and the structure and management of data within Firebase Firestore. This section emphasizes the seamless interaction between frontend and backend components.
- **Section 6: Test & Results** presents a detailed evaluation of the developed system's predictive capabilities with regression metrics, specifically RMSE, MAE, and  $R^2$ . It also includes the results of the application functionality testing, which is scenario-based reservation tests and their outcomes. Additionally, this section reports on the findings from usability assessments using the SUS, with additional questionnaire.
- **Section 7: Conclusion** synthesizes the main achievements and contributions of the project, highlighting the effectiveness and practical utility of the implemented EV charging recommendation and reservation system. It further suggests clear directions for future enhancements, highlighting opportunities for deeper integration and expansion with personalized user-centric features.

## 2 BACKGROUND

This section covers recent research across four key areas of EV charging systems. EV Charging Management and Reservation Systems focus on enabling users to efficiently reserve and manage charging sessions. Security and Privacy Based Reservation Systems address protecting user data and ensuring secure communication during charging and booking processes. Slot Recommendation and Scheduling Systems aim to optimize the selection and timing of charging slots to reduce waiting times and improve user experience. EV Charging Infrastructure Focus and Grid Optimization concentrates on balancing grid loads, minimizing peak demand, and enhancing the stability and efficiency of power distribution networks.

### 2.1 EV Charging Management and Reservation Systems

Vaidya et al.[4] proposed a smart electric vehicle charging management system designed to address the challenges of urban EV charging. They developed and implemented a solution named SecCharge (see Figure 2.1), which incorporates a reservation-based approach alongside a novel Multi-Objective Slot Allocation (MOSAL) method built on Multi-Objective Simulated Annealing (MOSA). The system optimizes charging schedules by balancing multiple factors, including waiting time, charging duration, monetary cost, station congestion, and user satisfaction. Through simulation experiments that modeled urban mobility scenarios using realistic EV specifications, the study demonstrated that SecCharge was able to lower average waiting times by around 20% when compared to traditional strategies, while also enhancing user satisfaction in high-demand conditions.

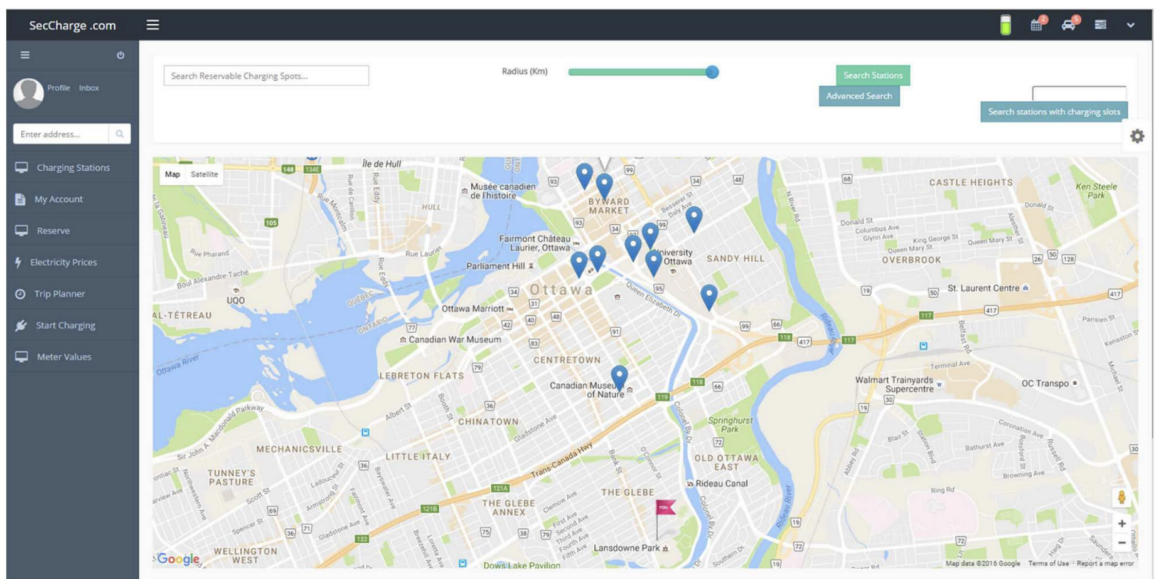


Figure 2.1: SecCharge user interface displaying nearby reservable EV charging stations on a map view. [4]

Kumar et al. [5] introduced S2RC (Smart Search of Route and Charging), an intelligent system aimed at enhancing EV route planning and charging slot reservation by dynamically incorporating real-time traffic conditions, charging station status, and vehicle energy consumption factors (see Figure 2.2). The system features a distributed architecture and leverages a multi-objective optimization framework that seeks to reduce energy use, minimize waiting times at charging stations, and lower overall charging costs. Evaluations performed using a realistic transportation network demonstrated that S2RC achieved up to a 3.4% reduction in energy consumption, shortened charging station wait times by as much as 69.3%, and decreased charging costs by up to 10.9% in comparison with conventional methods.

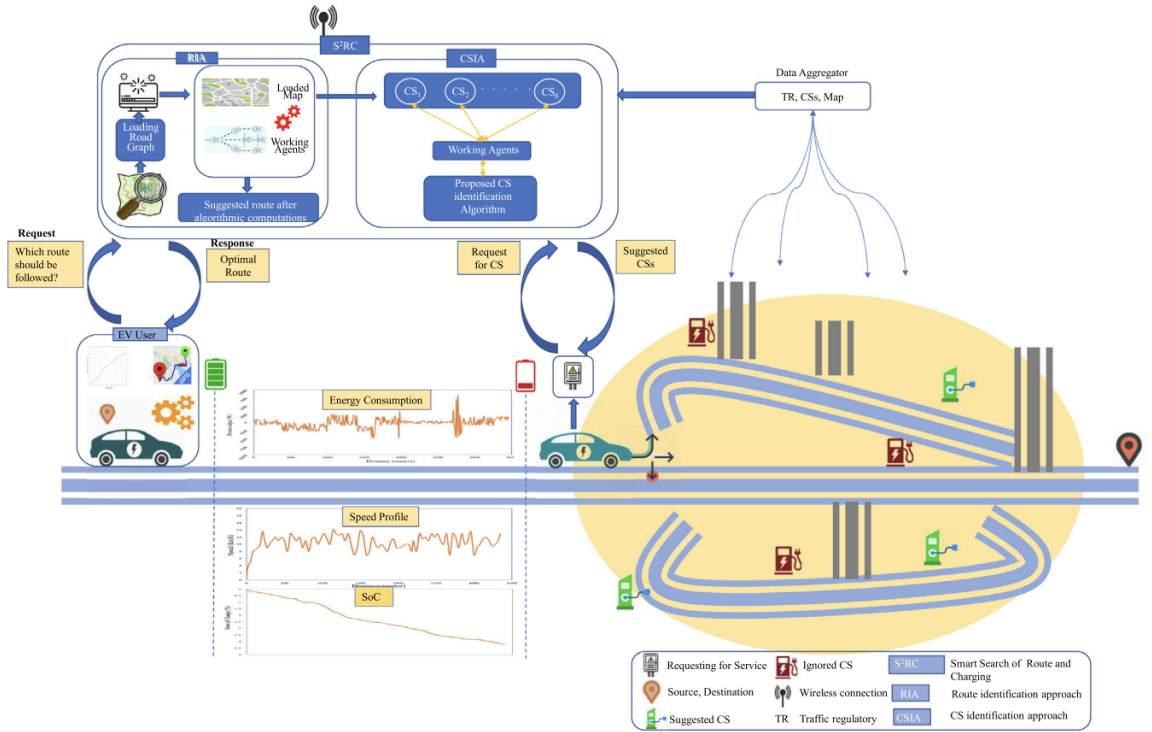


Figure 2.2: Illustrates the architecture of the S2RC system. [5]

Cao et al. [6] developed an EV charging management system aimed at optimizing the selection of charging stations for electric vehicles in motion, with a particular focus on minimizing total trip duration. The system (see Figure 2.3) leverages anticipated reservation information, integrates parking time considerations, and addresses uncertainties in vehicle mobility caused by varying traffic conditions through a mechanism of periodic reservation updates. Simulation experiments based on a realistic Helsinki city environment, involving 240 EVs and 7 charging stations, demonstrated that the approach significantly enhanced charging efficiency by reducing trip times and waiting periods, while also increasing the proportion of fully charged vehicles, thereby contributing to an improved user experience in dynamic urban contexts.

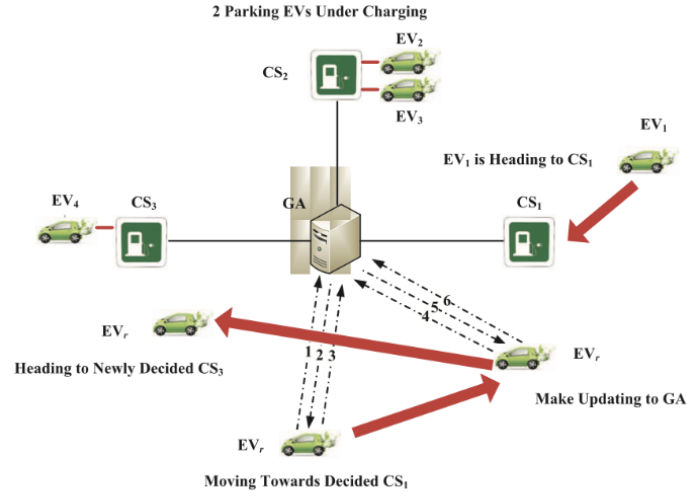


Figure 2.3: Overview of proposed EV charging management. [6]

Orcioni et al. [7] proposed an enhancement to the Open Charge Point Protocol (OCPP), extending its capabilities to include flexible EV charging slot reservations. Within this system, users can input their charging preferences—such as desired start time, session duration, target battery level, and acceptable cost range—through a mobile application interface (Figure 2.4). A central management system then evaluates these preferences and generates several optimized reservation options based on the user’s indicated flexibility. To assess system performance, the authors implemented a full prototype consisting of a web server, structured database, and an Android-based user app, followed by simulation tests using randomized reservation data. Findings revealed that when users were open to cost flexibility, the charging station’s revenue improved by 300%, while user satisfaction remained relatively high even at maximum station occupancy.

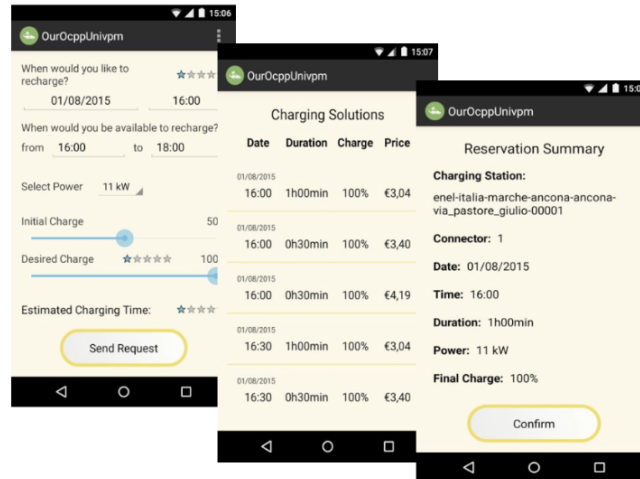


Figure 2.4: Images from the smartphone app of the system. [7]

Basmadjian et al. [8] developed an interoperable reservation system for public EV charging stations to improve infrastructure efficiency and user satisfaction. Their framework defines four reservation types: uncertain ad-hoc, guaranteed ad-hoc, uncertain planned, and guaranteed planned, and is based on the E-Mobility Systems Architecture (EMSA) (Figure 2.5). A prototype of the uncertain ad-hoc model was tested at four charging stations in Bavaria, Germany. Large-scale agent-based simulations with up to 3000 EV users showed that planned reservations effectively reduced trip durations and waiting times under high demand, while ad-hoc reservations were better suited to very high demand. Across all scenarios, station utilization remained below 10%, demonstrating the potential of interoperable reservation systems to enhance EV charging services in urban environments.

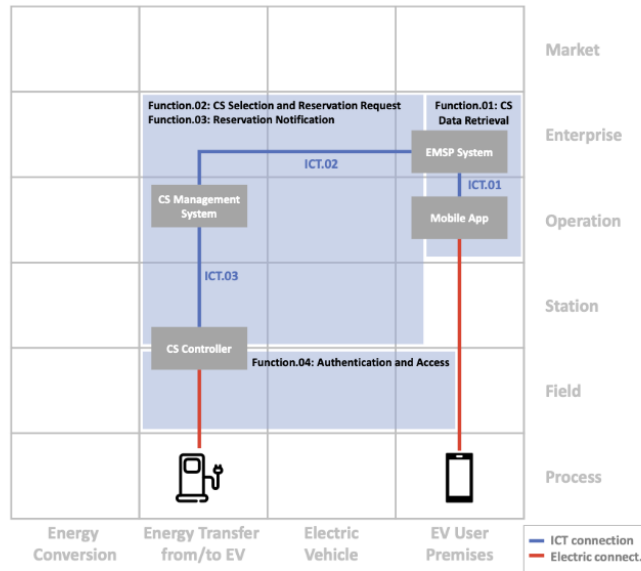


Figure 2.5: the system model for interoperable EV charging reservations based on the EMSA framework. [8]

Saqib et al. [9] developed a smart electric vehicle charging management solution based on cloud computing (see Figure 2.6), aimed at guiding users toward the most suitable charging stations by considering live factors such as pricing, congestion, and distance. The architecture connects various participants in the ecosystem including EV owners, charging service providers, and energy suppliers through a centralized cloud platform that enables features such as real-time bidding, dynamic load management, and integration with energy markets. A prototype system was implemented using ASP.NET for the front-end interface and C#.NET for backend processing, and its effectiveness was evaluated through simulation-based case studies. The findings showed that the platform successfully delivered up-to-date charging information, reduced user wait times, and supported economical energy trading, underscoring its potential to enhance EV charging networks and promote grid-friendly operation at scale.

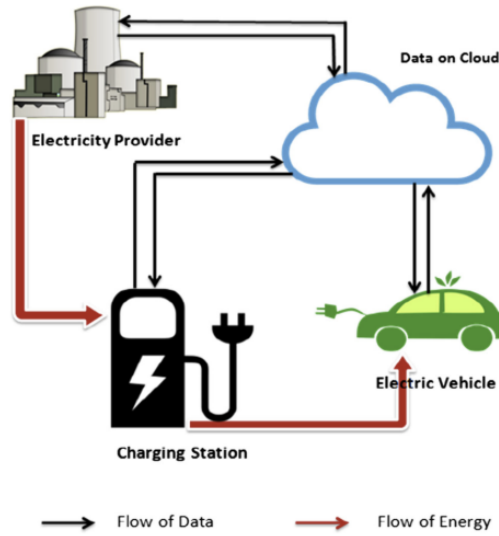


Figure 2.6: Illustrates the cloud-based EV charging architecture. [9]

Flocea et al. [10] developed a smart charging reservation algorithm for EV drivers to overcome the limitations of the Open Charge Point Protocol, which permits only immediate reservations. Their custom algorithm generates future reservation intervals based on past transaction data, predicting optimal time slots and preventing reservation conflicts, as illustrated in Figure 2.7. Integrated into the Smart EVC (Electrical Vehicle Charging) platform, the system was tested through multiple case studies, demonstrating that no overlapping reservations occurred and ensuring that only users with valid reservations could initiate charging. The solution successfully prevented queuing, optimized charging point availability, and improved overall user satisfaction.

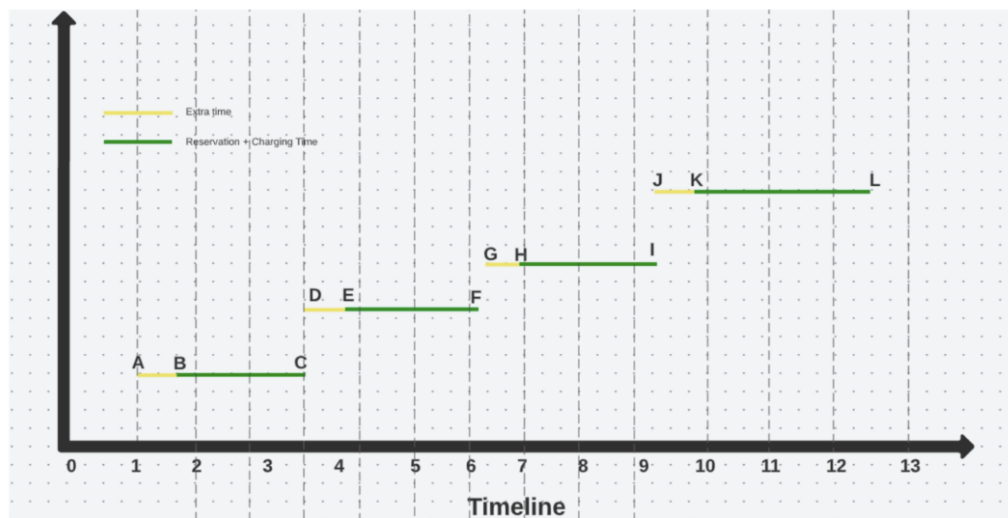


Figure 2.7: Illustrates the Smart EVC platform architecture. [10]

Eedara et al.[11] developed a smart EV charging management system that leverages machine learning to assist users in planning charging sessions during periods of lower grid demand and more affordable electricity rates. The system is trained on a dataset of 52,560 residential electricity consumption records to forecast both grid load patterns and expected electricity prices. After evaluating several machine learning algorithms, the Random Forest model delivered the most accurate results, achieving an  $R^2$  score of 0.89 for load prediction and 1.0 for price forecasting. The system also features a web-based platform where users can view upcoming pricing trends, make or cancel charging reservations, and receive timely updates. Additionally, it offers flexible reservation settings and built-in payment functionality. By encouraging off-peak charging, this approach supports improved grid efficiency and allows users to lower their overall charging costs.

Chavhan et al. [12] proposed a next-generation cyber-physical system for EVs as illustrated in Figure 2.8 that enables charging slot booking through EV-to-everything (EV2X) communication. Their system integrates roadside units (RSUs), Immediate Proximity Mobile Units (IPMUs), and a Global Aggregator (GA) to facilitate real-time charging reservations and optimize traffic flow at charging stations. They developed a stochastic M/M/m queuing model which represents a system with Markovian arrivals, Markovian service times, and multiple servers (m chargers), and formulated an optimization problem aimed at minimizing waiting times, costs, and travel distances. Large-scale simulations over a  $200 \times 200$  km<sup>2</sup> urban area with 50 charging stations and up to 25 EVs per station demonstrated that the proposed system could achieve up to 93.3% cost savings and reduce charging times by up to 40 minutes, validating its scalability and applicability for smart city deployment.

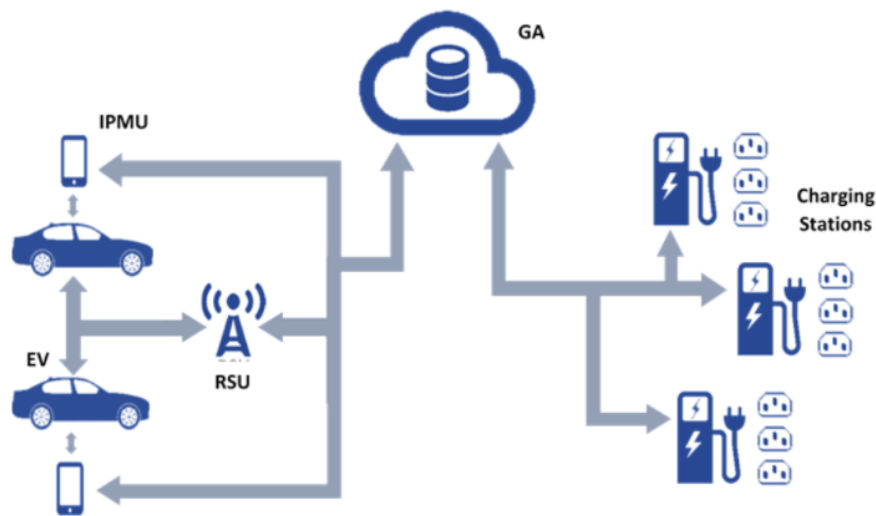


Figure 2.8: Illustrates the cloud-based EV charging architecture. [12]

## 2.2 Security and Privacy Based Reservation Systems

Cao et al. [13] Cao et al. introduced a blockchain-based EV charging reservation system designed to enhance the security, transparency, and efficiency of the reservation process. Built on Hyperledger Fabric, the system leverages smart contracts to manage reservations and payment transactions among distributed participants, including EV users, charging stations, and service operators (Figure 2.9). The framework allows users to view real-time information about station availability, congestion, and pricing, and to book charging slots in advance through a decentralized platform. To promote punctuality and fairness, the system incorporates a dynamic, behavior-aware pricing mechanism where users are rewarded for reliable reservation behavior and penalized for violations such as late arrivals or exceeding reserved energy amounts. The experimental evaluation showed that the platform maintained high throughput (220 transactions per second) with transaction latency kept below 500 milliseconds, while the behavior-based pricing contributed to shorter wait times and a more balanced use of charging infrastructure, demonstrating its potential to support a trustworthy and user-friendly charging ecosystem.

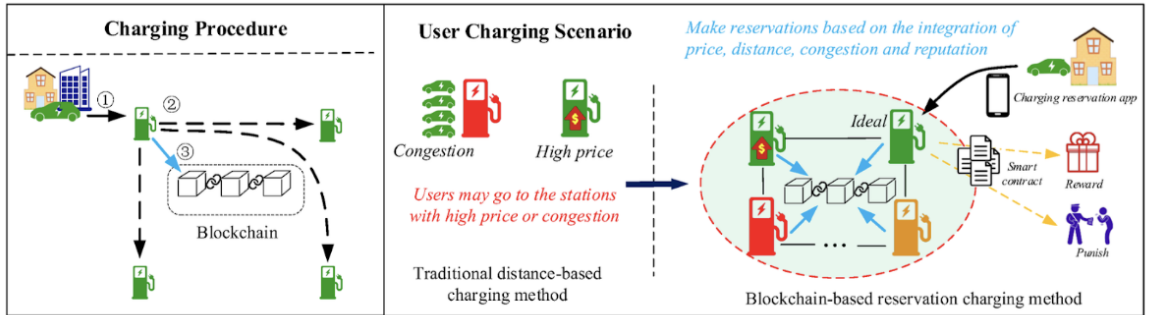


Figure 2.9: The reservation-based electric vehicle charging service mode. [13]

Liu et al. [14] developed a secure EV charging reservation framework designed to safeguard user privacy while ensuring efficient system performance. Their solution enables users to reserve charging slots anonymously, ensuring that sensitive information such as identity or location is not exposed during the process. This privacy-preserving capability is achieved through the use of advanced cryptographic methods, including anonymous authentication and zero-knowledge proofs (see Figure 2.10), which allow users to verify eligibility without revealing personal data. The design incorporates mechanisms to discourage misuse, including penalties for missed reservations and a cap on the number of concurrent bookings per user to promote fairness. A mobile prototype was implemented and tested through simulation and formal security validation, achieving reservation processing times of roughly 16 seconds on a standard smartphone. The evaluation confirmed the system's practical usability and its ability to uphold strong privacy protections in real-world scenarios.

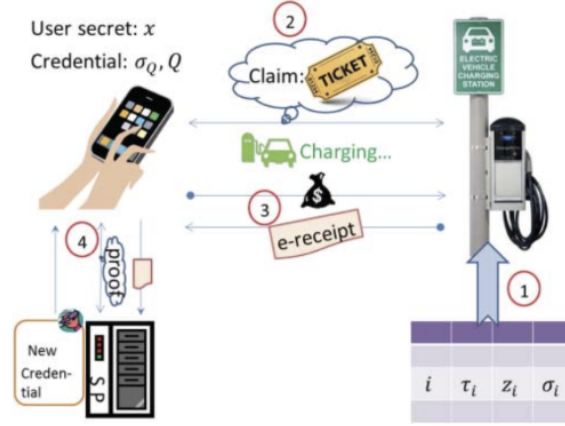


Figure 2.10: illustrates the privacy-preserving reservation protocol, where EV users securely reserve charging slots. [14]

Hamdare et al. [15] developed a hybrid EV charging management system (H-EVCMS) designed to enhance charging efficiency, balance grid load, and improve security across multiple charging stations. The system combines centralized optimization with distributed flexibility, enabling smart selection of stations, time-slot allocation, and dynamic pricing based on user preferences and peak demand periods. It also integrates advanced session management and cybersecurity features through an enhanced OCPP 1.6 protocol. Simulations comparing single-station and multi-station setups showed that H-EVCMS reduced load concentration, shifted demand to off-peak times, improved slot availability, and lowered charging costs. Resource utilization spread improved significantly from 10,888 units in the single-station scenario to 7,711 units in the multi-station case demonstrating the framework's scalability and efficiency for large-scale EV charging deployments.

### 2.3 Slot Recommendation and Scheduling Systems

Timpner et al. [16] developed a coordinated scheduling framework for EV charging within the V-Charge project, which combines autonomous valet parking with intelligent management of limited charging infrastructure in large-scale parking environments. The study explored five distinct scheduling algorithms Scanning, Shifting, Flexible, Earliest Deadline First (EDF), and Greedy spanning both static and dynamic allocation methods. Simulations using empirical parking data from Hamburg Airport and the City of Braunschweig were conducted to assess the performance of each approach. Results indicated that static algorithms, particularly Scanning, Shifting, and Flexible, consistently satisfied over 99% of parking and charging requirements and achieved high utilization rates when adequate charging capacity was available. In contrast, the dynamic methods (EDF and Greedy) exhibited lower efficiency in resource use, leading to reduced system performance.

Zhang et al. [17] proposed Master, an intelligent EV charging recommendation framework based on multi-agent reinforcement learning, aimed at addressing key challenges in urban EV charging such as long waiting times, charging inefficiencies, and service disruptions. In this system, each charging station is modeled as an independent agent that learns optimized recommendation policies by considering both current station availability and future demand patterns. The framework adopts a multi-agent actor-critic architecture with a centralized attentive critic to promote cooperation among stations and incorporates a delayed access strategy to account for future charging competition during training. Additionally, Master employs a dynamic multi-critic optimization method to jointly minimize average charging wait time, charging cost, and charging failure rate. The model was trained and evaluated using real-world datasets from Beijing and Shanghai, covering over 150,000 charging requests. On the Beijing dataset, experimental results showed that Master reduced average wait times by 51.4%, decreased charging failures by 96.5%, and lowered average charging costs by 13.6% compared to conventional baselines, demonstrating significant improvements in overall system performance and user experience.

Liu et al. [18] presented a reservation-based EV charging recommendation system that incorporates an Urgency First Charging (UFC) strategy, which assigns priority to vehicles with greater energy needs and limited remaining parking time. To optimize overall travel efficiency, they designed a charging station selection method that dynamically accounts for current congestion levels and allows priority-based preemptive charging. The system was tested through simulations in a realistic urban environment modeled on Helsinki, featuring 660 electric vehicles and extended parking durations (3600 seconds). Results demonstrated that the proposed approach significantly enhanced system performance, boosting the number of fully charged vehicles by 59%, reducing incomplete charges by 61%, lowering average waiting times by 14.6%, and shortening trip durations by 12.2%, thus offering tangible improvements in service quality and efficiency.

Ibrahim et al. [19] developed an intelligent EV charging recommendation system that helps users identify optimal charging stations by analyzing individual preferences, station availability, and historical usage patterns. The framework integrates a Restricted Boltzmann Machine (RBM) with the Waterwheel Plant Algorithm (WWPA), which fine-tunes the RBM's parameters to enhance prediction accuracy. The system was trained on a real-world dataset of 3,395 EV charging sessions collected from 105 stations across multiple workplace locations. It takes into account various user-centric factors such as charging speed, cost, network compatibility, and station proximity. To validate performance, the authors compared the WWPA-RBM model against several baseline RBM-based models using advanced regression metrics and statistical tests. The proposed model achieved superior results, with an RMSE of 0.0059, MAE of 0.0046, and an  $R^2$  of 0.991, significantly outperforming other approaches.

These findings demonstrate the system’s ability to deliver highly accurate, personalized charging recommendations, thereby enhancing user experience and helping to alleviate range anxiety.

Suanpang et al. [20] proposed a multi-agent reinforcement learning (MARL)-based approach to enhance EV charging station recommendations in smart city environments, with the goal of minimizing driver waiting times, reducing charging costs, and lowering the likelihood of service failures. In this architecture, each charging station operates as an autonomous agent that collaboratively learns effective recommendation policies. The study conducted a comparative analysis of several MARL algorithms, including Deep Q-Network (DQN), Deep Deterministic Policy Gradient (DDPG), and Multi-Agent DDPG (MADDPG), using empirical urban datasets. The MADDPG algorithm delivered the most favorable outcomes, reducing the mean charging wait time to 14.54 minutes, decreasing the failure rate to 3.86%, and yielding a cost savings of 12,305 units demonstrating its potential to support the development of smarter, more user-centric EV charging services.

## 2.4 EV Charging Infrastructure Focus and Grid Optimization

Shibl et al. [21] developed a deep reinforcement learning system (Figure 2.11) to manage EV charging while considering fast charging, conventional charging, vehicle-to-grid operations, and battery degradation. The system aims to protect the distribution grid by reducing voltage fluctuations, power losses, and transformer overloads, while also lowering user charging costs. In their model, EV chargers act as the environment, and EV users are the learning agents that adapt their charging strategies through interaction. The researchers tested the system on multiple real-world case studies, including scenarios with  $\pm 20\%$  uncertainty in electricity prices. The results showed that the system successfully smoothed load curves, reduced maximum voltage fluctuation from 0.17 to 0.07, and maintained reliable performance under uncertain conditions, proving its scalability and grid-friendly behavior.

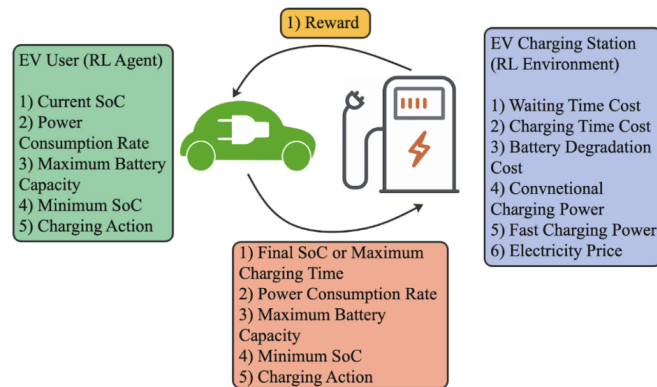


Figure 2.11: EV charging station management solution overview. [21]

Mehta et al. [22] proposed two smart charging strategies for plug-in electric vehicles (PEVs), designed to optimize both grid-to-vehicle (G2V) and vehicle-to-grid (V2G) operations within existing distribution system infrastructure. The first strategy minimizes total daily charging costs, while the second reduces the peak-to-average load ratio (PAR) to improve grid stability. Both approaches were implemented using a genetic algorithm with heuristic initialization and tested on an 11 kV 37-bus industrial distribution network model. The framework was applied to workplace charging scenarios, where PEVs remain connected during daytime hours and can participate in both charging and discharging operations. Simulation results showed that the cost-focused strategy supported up to 10% PEV penetration without violating transformer and line loading constraints, while the PAR-focused strategy allowed up to 60% penetration. Furthermore, the PAR-based strategy achieved a peak load increase of just 3.86% and produced a flatter load profile, demonstrating its superior scalability and suitability for accommodating growing numbers of PEVs without requiring costly infrastructure upgrades.

Aoun et al. [23] introduced PONDCOA (Proof of Need based Dynamic Charging Optimization Algorithm), a dynamic EV charging optimization method designed to improve charging schedules while mitigating peak demand on the power grid. PONDCOA applies a novel "Proof of Need" (PoN) mechanism to prioritize charging based on several key factors, including battery state of charge, charger capacity, and the number of EVs connected per household. The system dynamically adjusts charging decisions in real time, ensuring that vehicles with the most urgent needs are prioritized without exceeding grid capacity limits. The algorithm is specifically designed to operate in decentralized, end-of-line distribution networks and integrates advanced control and communication mechanisms at the substation level. To validate its effectiveness, simulations were conducted involving 30 EVs under various grid constraint scenarios. Results demonstrated that PONDCOA reduced peak charging demand by 40.8% compared to uncoordinated charging, while maintaining the total charging duration at 10 hours equal to the uncoordinated baseline thereby confirming its ability to balance grid load without compromising user satisfaction.

Yang et al. [24] proposed a Block Model Predictive Control (BMPC) method to improve EV charging in commercial settings, where demand charges can make electricity more expensive. They described the charging schedule as a Markov Decision Process (MDP) and created a control system that balances short-term charging decisions with long-term cost management by spreading demand charges across the schedule. They tested their approach using real data from the Adaptive Charging Network (ACN) testbed and compared it to four other well-known methods. The results showed that BMPC improved total rewards by up to 12% compared to the other methods and stayed within 1% of the best possible performance, even when traffic patterns changed and forecast errors were as high as 20%, showing that the method is reliable and effective.

### 3 METHODOLOGIES

This section presents the core algorithms, software tools, datasets, and evaluation methods applied in the development of intelligent EV charging management and reservation systems. It highlights the technical components that enable system functionality and performance validation through simulations and real-world data analysis.

#### 3.1 Regression Algorithms

Regression algorithms are supervised machine learning techniques designed to model the relationship between one or more input variables (features) and a continuous target variable (see Figure 3.1). Unlike classification algorithms, which assign data points to discrete categories, regression models aim to predict a value that can take any number within a defined range. These algorithms are particularly useful when the objective is to estimate quantities that vary over time or under different conditions such as temperature, energy consumption, or, as in this study, the availability of charging stations. During the training process, a regression model learns from historical data by minimizing the error between its predicted outputs and the actual observed values, often using loss functions such as Mean Squared Error (MSE) or Mean Absolute Error (MAE). Once trained, the model can generalize from this knowledge to provide reliable predictions on new, unseen data, making regression a powerful tool for forecasting, optimization, and decision support in real-world applications.

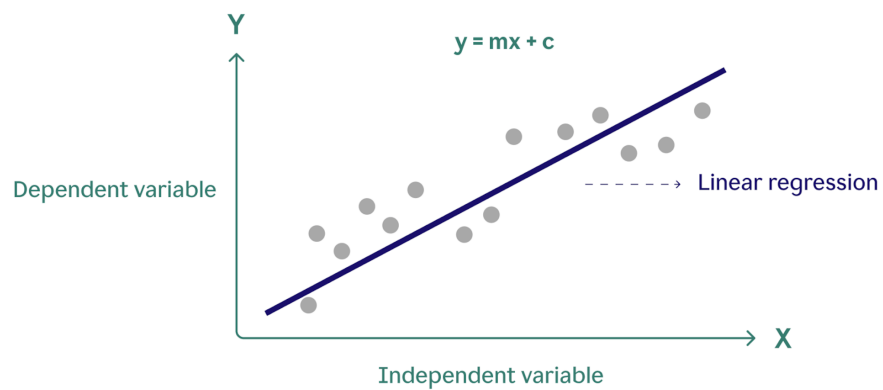


Figure 3.1: Visualization of linear regression [25].

Regression models can be broadly categorized based on how they capture the relationship between input features and the target variable, particularly in terms of linearity and non-linearity. One major category is linear regression models, which assume that the relationship

between the input variables and the target is linear—meaning that changes in the inputs produce proportional changes in the output. For example, in a simple linear regression model predicting a person’s weight based on their height, each additional centimeter of height might be associated with a constant increase in expected weight. These models aim to fit a straight line or a linear combination of features to predict outcomes. Linear models are simple, interpretable, and computationally efficient, and they perform well when the underlying data structure exhibits linear relationships. However, they are limited in their ability to model complex interactions or non-linear trends unless significant feature engineering or transformations are applied.

Non-linear regression models do not assume a simple, straight-line relationship between input features and the target. In other words, a change in one feature does not always cause the same amount of change in the output. The effect can vary depending on the situation(see Figure 3.2), and the pattern can be curved or irregular. For example, in EV charging data, the number of available charging spots may drop quickly during morning rush hours, stay more stable in the afternoon, and decrease again in the evening. This is not a straight-line pattern it is non-linear. Likewise, charging demand can behave very differently on weekends compared to weekdays, creating non-linear patterns across the week.

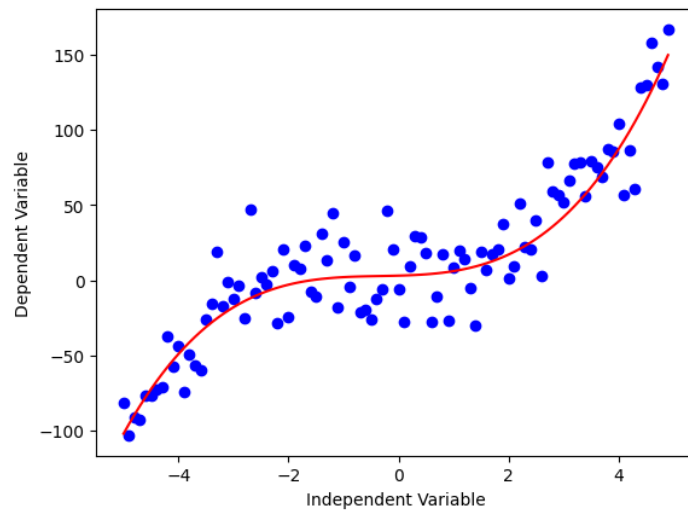


Figure 3.2: A visual representation of a Non-Linear Regression [26].

Tree-based regression models are a common type of non-linear model. Instead of trying to fit a single formula to the data, these models split the data into smaller groups using simple decision rules. The model builds a "tree" structure, where each decision (such as "is time of day before 9 AM?") splits the data. At the end of each branch, the model makes a prediction. This way, tree-based models can easily handle complex relationships and sudden changes in the data. They are also practical: they can work with both numbers and categories, are not

too sensitive to outliers, and their decisions are easy to understand. These strengths make them a good choice for modeling the changing and sometimes unpredictable patterns seen in EV charging station availability.

### 3.1.1 Decision Tree Regression

**Decision Tree Regression** is a tree-based machine learning algorithm used to predict continuous (numerical) target values. The model divides the data into branches through a series of decision nodes based on the features (see Figure 3.3). At each node, the data is split according to a threshold value of a particular feature; this process continues until further splitting no longer provides meaningful improvement or until predefined stopping criteria are met. These criteria may include a maximum tree depth, a minimum number of samples required in a node, or a minimum required reduction in error for a split to be accepted. In the resulting leaf nodes, the model assigns the average target value of the samples that fall into each leaf as the prediction. When a new instance is introduced, the model traverses the tree structure to generate the corresponding prediction. This method can effectively model complex and non-linear relationships, and the learned rules are visually interpretable as a tree, which makes the model easy to understand. Its advantages include not requiring feature scaling, the ability to handle both categorical and numerical data, and high interpretability. However, its key disadvantages are its sensitivity to small changes in the data (instability) and its tendency to overfit when the tree becomes too deep. These drawbacks arise from the model's flexibility and its tendency to fit the training data too closely by creating an overly complex tree structure.

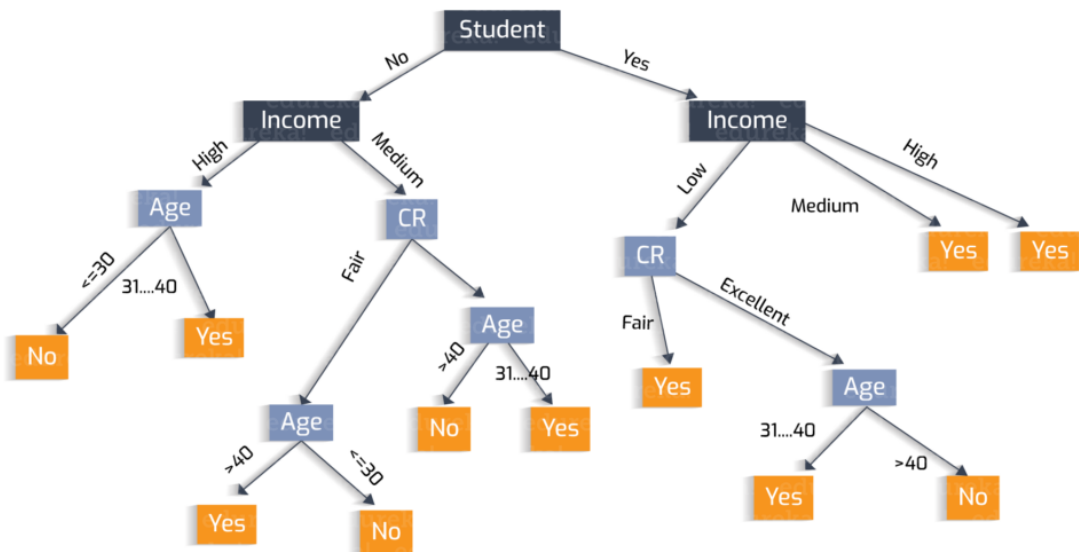


Figure 3.3: Decision Tree Regression [27].

### 3.1.2 Random Forest Regression

**Random Forest Regression** is a robust ensemble method used for predicting continuous numerical values. It builds multiple decision tree regressors (see Figure 3.4), each trained on a bootstrap sample a randomly selected subset of the original training data with replacement. This strategy ensures diversity among the trees by exposing each one to slightly different data. Furthermore, at every decision node in a tree, the model selects the best split from a random subset of features, not from the full feature set, introducing additional randomness and reducing correlation among the trees.

To illustrate how this randomness enhances model performance, consider a dataset with ten input features. In a traditional decision tree, each split is determined by evaluating all available features to find the one that minimizes the prediction error. However, in Random Forest Regression, at each decision node, only a randomly selected subset of features (e.g., three out of ten) is considered for splitting. This controlled randomness ensures that individual trees follow different structural paths, even when trained on similar data. As a result, trees become decorrelated and capture diverse patterns within the dataset. This diversity prevents the ensemble from overfitting and allows the Random Forest to learn more generalizable relationships rather than over-specializing to the training data.

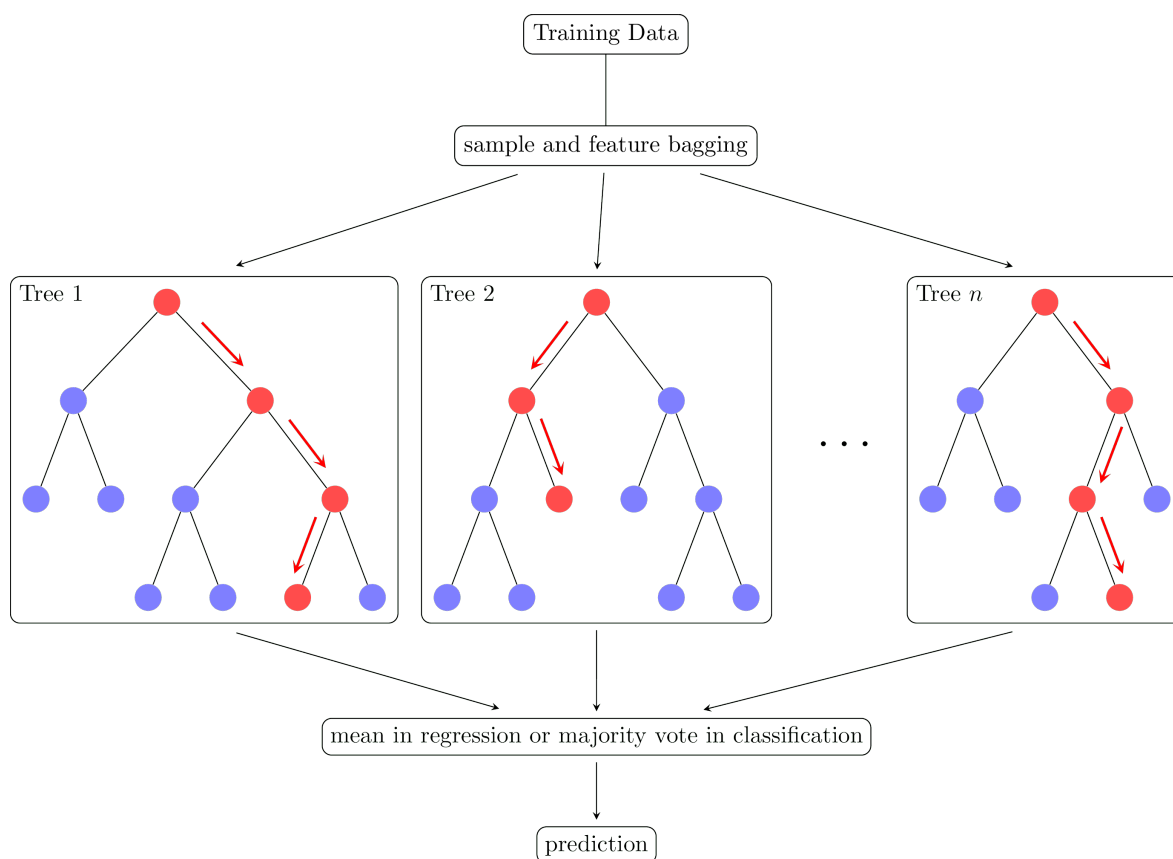


Figure 3.4: Random Forest Regression [28].

Each decision tree can grow to a different depth and have a unique structure, since the data and features they use are different. The growth of a tree typically stops when certain conditions are met such as reaching a maximum depth, achieving a minimum number of samples in a node, or when further splits offer negligible improvement. Unlike standalone trees, Random Forest often uses fully grown trees because the ensemble approach offsets the risk of overfitting.

Once all trees are built the final prediction is made by averaging the outputs of all trees. This averaging mechanism reduces variance, improves robustness, and results in a model that generalizes well. The strength of Random Forest lies in combining diverse learners to form a more balanced and accurate prediction model than any individual tree.

### 3.1.3 Gradient Boosting Regression

**Gradient Boosting Regression** Gradient Boosting Regression is a highly effective ensemble learning technique used for predicting continuous (numerical) target values. Unlike methods that build decision trees independently, Gradient Boosting constructs them sequentially, with each tree aiming to correct the residual errors made by the combined predictions of the earlier trees (see Figure 3.5). This step-by-step learning process is guided by the gradients of a loss function, which point the algorithm in the direction of greatest error reduction hence the term "gradient boosting."

At each iteration, the algorithm calculates how much the current predictions differ from the actual values (i.e., the residuals), and then trains a small decision tree to model those residuals. The predictions from this new tree are scaled (often by a small learning rate) and added to the previous output, gradually improving model performance. Over many iterations, this allows the model to reduce bias, learn from complex patterns, and refine predictions with increasing precision.

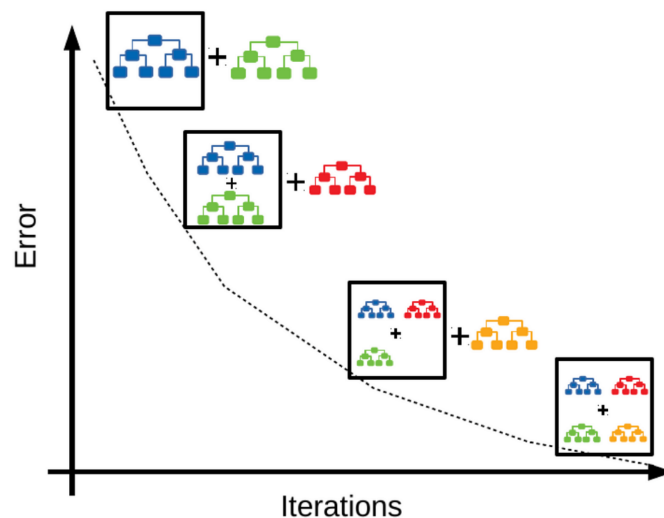


Figure 3.5: Gradient Boosting Regression [29].

A key difference between Gradient Boosting and Random Forest Regression lies in how the trees are constructed and combined. While Random Forest grows many trees independently and in parallel, averaging their outputs to reduce variance and prevent overfitting, Gradient Boosting trains trees sequentially, where each tree's output builds directly upon the last. This makes Gradient Boosting especially effective at targeting difficult-to-predict patterns, but also more sensitive to errors and noise in the training data.

These benefits come with important trade-offs. Gradient Boosting is more sensitive to hyperparameter settings, such as the learning rate, number of trees, and tree depth. Poor configuration can lead to overfitting or underperformance. Additionally, since each tree must wait for the previous one to finish, training is slower and less parallelizable than with Random Forests.

In summary, Gradient Boosting Regression often delivers superior predictive accuracy, especially on structured and complex datasets, by iteratively correcting its own mistakes. However, it demands more computational resources, careful tuning, and longer training times, whereas Random Forest offers a faster, simpler, and more robust alternative.

## **3.2 Software & Tools**

This subsection presents the essential software tools and development environments used throughout the design, implementation, and testing phases of EV charging applications. It covers mobile development platforms like Xcode and Swift, backend integration tools such as Firebase and Google APIs, and data analysis environments like Google Colab. Together, these tools support seamless frontend-backend integration, real-time data processing, and predictive modeling essential for intelligent EV charging systems.

### **3.2.1 iOS Native**

iOS native development involves building applications specifically tailored for Apple devices using tools and languages provided by Apple. One of the most important tools in this ecosystem is Xcode, shown in Figure 3.6. Xcode serves as Apple's official development environment, enabling developers to design, code, test, and debug their apps all in one place. It comes with features like a built-in code editor, simulators for different devices, performance monitoring tools, and visual interface builders such as Storyboard or SwiftUI.

The primary programming language used for iOS app development is Swift. Swift is a modern, open-source language created by Apple to replace Objective-C. It was designed to be easy to read and write, while also being fast and reliable. Swift includes features like optionals, closures, generics, and automatic type detection, which make code more concise and help reduce common programming errors.

In addition to the Swift language itself, Apple introduced SwiftUI, a newer framework for building user interfaces. Unlike older approaches where the developer had to define every UI update manually, SwiftUI uses a declarative style. This means developers can describe what the interface should look like, and the system automatically keeps it updated when the app's data changes. SwiftUI is especially useful for creating responsive and interactive layouts, and it reduces the need for repetitive code compared to traditional UIKit development. Together, Xcode, Swift, and SwiftUI provide a complete and efficient environment for building smooth and visually appealing iOS applications.

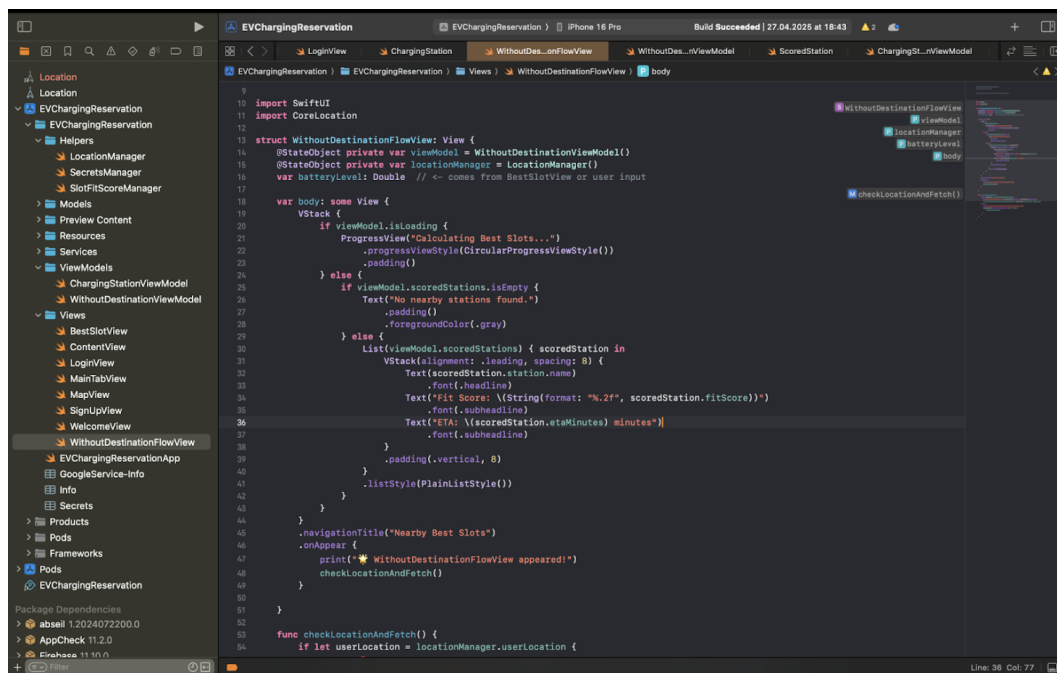


Figure 3.6: Xcode Environment.

### 3.2.2 Google APIs & Firebase Console

Google provides a rich set of APIs and services that help developers add advanced features to their applications easily. These services work through APIs, allowing apps to connect to Google's systems without needing complex custom code. Most of Google's APIs, including Maps, Directions, and Firebase, follow a pay-as-you-go pricing model, where developers only pay based on the resources they actually use.

One widely used tool is the Google Maps API, which lets developers embed interactive maps into their applications. It can show detailed information such as roads, landmarks, user locations, and more. This is useful for apps that rely on geographic data or navigation.

The Google Directions API is another key service. It helps calculate routes between locations and provides useful data like distance, estimated travel time, and current traffic conditions. This information can be used to guide users efficiently from one point to another.

In addition to these APIs, Firebase is a cloud-based platform offered by Google that simplifies backend development. Through the Firebase Console (see Figure 3.7), developers can manage core services such as databases, user authentication, and app analytics.

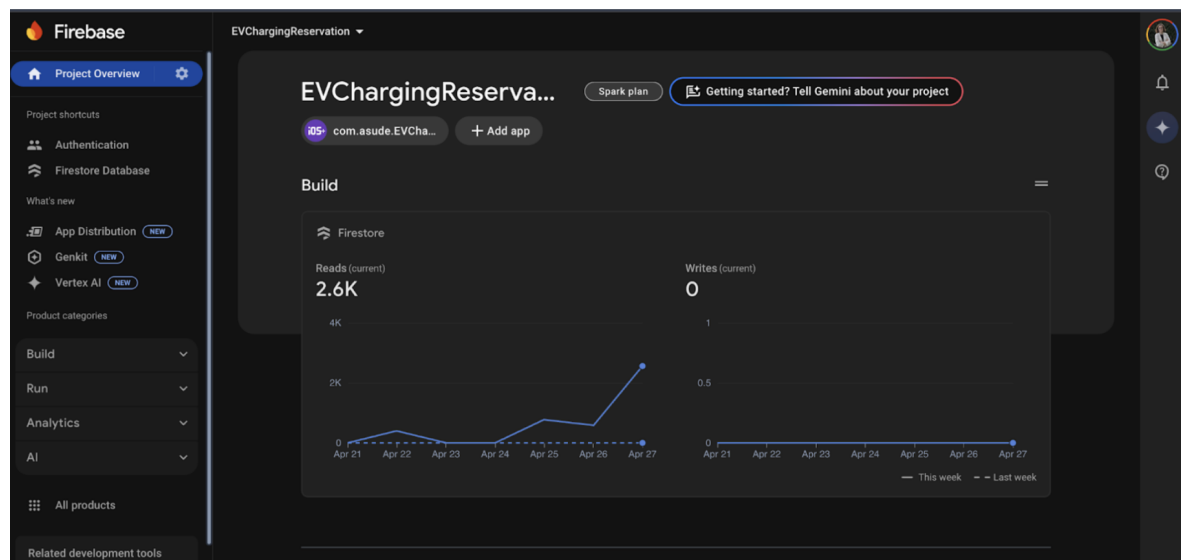


Figure 3.7: Firebase Console.

Key Firebase features include:

- **Authentication:** Allows developers to handle secure login and user management using various methods like email/password, phone number, or social login providers (e.g., Google or Apple). It supports session tracking and password recovery out of the box.
- **Cloud Firestore:** A scalable, real-time NoSQL database that organizes data into documents and collections. It supports live data synchronization across devices, which is ideal for applications needing instant updates.

Together, Google APIs and Firebase offer a flexible and scalable environment for building responsive, data-driven mobile and web applications.

### 3.2.3 Google Colab

Google Colab is a cloud-based Python development environment provided by Google, allowing users to write, run, and share code directly in their web browsers without requiring any local setup. It's especially useful for tasks like data analysis, machine learning, and academic research because it removes the need for software installation or hardware configuration. Users can take advantage of Google's powerful computing resources, including free access to GPUs and TPUs, which can significantly speed up complex computations and the free usage limit is often well enough for most educational tasks, exploratory analyses, and small to mid-sized data projects.

As shown in Figure 3.8, the Google Colab environment allows users to carry out essential data inspection tasks directly within the notebook, such as checking for missing values or generating quick statistical summaries using built-in functions. These basic checks help establish a clear starting point for deeper analysis.

```

import pandas as pd

train_df = pd.read_csv('/train.csv')
test_df = pd.read_csv('/test.csv')

# Display the first few rows of the training set
print(train_df.head())

# Check for missing values
print(train_df.isnull().sum())

# Get summary statistics
print(train_df.describe())

```

|   | date                | Station           | Available | Charging | Passive | \ |
|---|---------------------|-------------------|-----------|----------|---------|---|
| 0 | 2020-07-03 00:00:00 | FR+V75+EBELI*1*1  | 3         | 0        | 0       |   |
| 1 | 2020-07-03 00:00:00 | FR+V75+EBELI*10*1 | 2         | 1        | 0       |   |
| 2 | 2020-07-03 00:00:00 | FR+V75+EBELI*11*1 | 2         | 1        | 0       |   |
| 3 | 2020-07-03 00:00:00 | FR+V75+EBELI*12*1 | 3         | 0        | 0       |   |
| 4 | 2020-07-03 00:00:00 | FR+V75+EBELI*13*1 | 3         | 0        | 0       |   |

|   | Other | tod | dow | trend | Latitude  | Longitude | Postcode | area  |
|---|-------|-----|-----|-------|-----------|-----------|----------|-------|
| 0 | 0     | 0   | 6   | 27    | 48.855667 | 2.354089  | 75004    | south |
| 1 | 0     | 0   | 6   | 27    | 48.864240 | 2.397724  | 75020    | east  |
| 2 | 0     | 0   | 6   | 27    | 48.861016 | 2.283080  | 75016    | west  |
| 3 | 0     | 0   | 6   | 27    | 48.824388 | 2.336857  | 75014    | south |
| 4 | 0     | 0   | 6   | 27    | 48.882640 | 2.335728  | 75009    | north |

|   | date | Station | Available | Charging | Passive | Other | tod | dow | trend | Latitude |
|---|------|---------|-----------|----------|---------|-------|-----|-----|-------|----------|
| 0 | 0    | 0       | 0         | 0        | 0       | 0     | 0   | 0   | 0     | 0        |

Figure 3.8: Colab Environment.

For more advanced exploration, analysis, and data processing, Colab supports a variety of widely used Python libraries that simplify and accelerate the workflow. Pandas is commonly used for handling structured data, offering tools to load datasets, clean and transform them, and perform group-based operations which means splitting the data into subgroups (e.g., by day, category, or user) and applying calculations like averages, counts, or sums within each group. NumPy is another core library, designed for fast numerical computations, especially with arrays and matrices, making it ideal for scientific and mathematical tasks. For visualizing data, Matplotlib provides a flexible way to create plots, charts, and graphs, helping users spot trends and patterns more easily. These libraries work seamlessly together within Colab, forming a powerful toolkit for data-driven projects.

One of Colab's standout features is real-time collaboration, multiple users can edit and comment on a notebook at the same time. Notebooks are saved directly to Google Drive, so all progress is automatically backed up and accessible from anywhere. Colab also supports easy integration with external data sources like Google Drive, GitHub, or public URLs, giving users the flexibility to work with real datasets in live environments. This combination of accessibility, power, and collaboration makes Google Colab a popular choice for both beginners and experienced developers working on Python-based data projects.

### 3.3 Datasets

This section presents the datasets used to support the analysis, modeling, and evaluation of EV charging station management strategies. The datasets provide detailed historical and spatial information about charging station utilization and electric vehicle charging behaviors. They are essential for understanding temporal patterns, predicting future charging demand, optimizing infrastructure use, and developing smart reservation and management systems. Each dataset is briefly introduced with a focus on its structure, key features, and relevance to EV charging applications.

#### 3.3.1 Paris Charging Station Dataset

The Paris Charging Stations Dataset, featured in the Smarter Mobility Data Challenge, offers a rich historical record of EV charging activity across the city. It includes over 1.85 million entries collected from 91 stations each equipped with 3 plugs between July 3, 2020, and February 18, 2021. Data is captured at 15-minute intervals and records four key metrics at each time step:

- **Available:** The number of charging plugs that are free and available
- **Charging:** The number of plugs currently in use for active charging
- **Passive:** The number of plugs connected to a vehicle but not actively charging (idle)
- **Other:** The number of plugs that are offline or out of order

As illustrated in Figure 3.9, stations are geographically spread across Paris and grouped into four regions: East, North, West, and South. This spatial segmentation supports analysis of local charging patterns and infrastructure performance. The dataset's detailed, time-series nature makes it an excellent foundation for studying plug availability, understanding usage trends, and developing predictive models for smarter EV charging strategies and urban planning.



Figure 3.9: Distribution of charging stations. [1]

### 3.3.2 ST-EVCDP Dataset

The ST-EVCDP (Spatio-Temporal Electric Vehicle Charging Demand Prediction) dataset is a comprehensive, real-world dataset created to facilitate research on EV charging demand and urban energy optimization. Collected from Shenzhen one of China’s most densely populated and technologically advanced cities the dataset spans a 30-day period from June 19 to July 18, 2022. It provides highly granular data at 5-minute intervals, resulting in 8,640 time steps and covering usage patterns from 18,061 public EV charging piles distributed throughout the city. As illustrated in Figure 3.10, the spatial layout of the charging network reveals uneven distribution patterns, reflecting differences in infrastructure density across various districts.

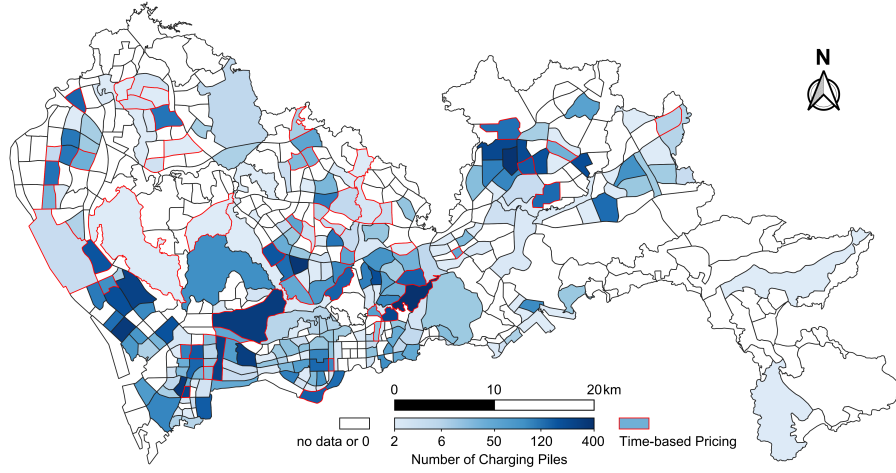


Figure 3.10: Geographical distribution of EV charging piles in Shenzhen. [30]

To represent the city’s layout more effectively, the dataset is structured as a graph with 247 nodes (traffic zones) and 1,006 edges representing neighboring relationships between them. This graph-based format is ideal for developing spatiotemporal machine learning models, as it captures not only the temporal evolution of charging demand but also the spatial dependencies between zones. Such a setup enables deeper analysis of how charging behaviors are influenced by urban layout and traffic flow. With its fine temporal resolution and spatial structure, ST-EVCDP is a valuable resource for modeling urban EV demand, designing predictive systems, and informing infrastructure development for sustainable smart cities.

## 3.4 Test Methodologies

This subsection presents the testing methodologies applied to evaluate both the prediction model, using metrics such as Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and R-squared ( $R^2$ ) and the overall usability of the application, assessed through the System Usability Scale (SUS) test.

### 3.4.1 Evaluation Metrics

Root Mean Squared Error (RMSE) is a commonly used metric for evaluating the performance of regression models, as it measures how well the model's predictions match the actual values. In other words, it shows the average magnitude of the prediction errors, the differences between the values predicted by the model and the true observed values. RMSE is calculated by first computing the difference between each true value and its corresponding predicted value, squaring these differences to eliminate negative values and to give more weight to larger errors, then taking the mean of these squared differences, and finally taking the square root of this mean. The formula for RMSE is given in Equation 1.

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (1)$$

where  $n$  is the number of data points,  $y_i$  is the true value for the  $i^{\text{th}}$  data point, and  $\hat{y}_i$  is the predicted value.

Since RMSE is expressed in the same units as the target variable, the size of the error can be directly understood in the context of the problem. Another key feature of RMSE is that it gives more weight to larger errors, because the differences between predicted and actual values are squared before averaging. This makes RMSE especially useful in cases where large prediction errors are more costly or unacceptable. The RMSE value is always non-negative, and a value of zero would indicate perfect predictions. While RMSE does not have a fixed maximum value, its range depends on the scale of the target variable, lower RMSE values always indicate better model accuracy. For these reasons, RMSE is commonly used both to evaluate a single model's performance and to compare different models, helping guide model selection and tuning.

Mean Absolute Error (MAE) is another widely used metric for evaluating the performance of regression models, as it measures the average absolute difference between the model's predictions and the actual observed values. In other words, MAE indicates how much, on average, the model's predictions deviate from the true values, without considering the direction of the error (whether the prediction was too high or too low). MAE is calculated by taking the absolute value of the difference between each true value and its corresponding predicted value, summing these absolute differences across all data points, and then dividing by the number of data points. The formula for MAE is shown in Equation 2.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (2)$$

where  $n$  is the number of data points,  $y_i$  is the true value for the  $i^{\text{th}}$  data point, and  $\hat{y}_i$  is the predicted value.

One of the key characteristics of MAE is that it treats all errors equally, as it does not square the errors like RMSE does. This means that MAE is less sensitive to large individual errors and provides a more balanced view of the average prediction error. Like RMSE, MAE is expressed in the same units as the target variable, making it easy to interpret in the context of the problem. A value of zero would indicate perfect prediction accuracy. In addition to being used as an evaluation metric, MAE is also frequently used as a loss function during model training, particularly in applications where minimizing the average size of the prediction errors is more important than penalizing large errors more heavily. Because MAE provides a straightforward and robust measure of model accuracy, it is often used alongside other metrics, such as RMSE and  $R^2$ , to provide a more comprehensive evaluation of a model's predictive performance.

R-squared ( $R^2$ ), is a widely used and important metric for evaluating the goodness of fit of a regression model. Unlike RMSE or MAE, which directly measure the size of the prediction errors,  $R^2$  provides a relative measure of how well the model explains the variance in the target variable. In other words, it indicates what proportion of the variability in the actual values can be explained by the model's predictions. The formula for  $R^2$  is shown in Equation 3.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (3)$$

where  $n$  is the number of data points,  $y_i$  is the true value for the  $i^{\text{th}}$  data point,  $\hat{y}_i$  is the predicted value, and  $\bar{y}$  is the mean of the actual values. The numerator represents the sum of squared errors of the model (also known as residual sum of squares), while the denominator represents the total variance in the target variable (total sum of squares).

$R^2$  values range between negative infinity and 1. A value of 1 indicates a perfect model that explains all of the variance in the target variable, while a value of 0 means that the model performs no better than simply predicting the mean of the target variable. Negative  $R^2$  values indicate that the model performs worse than predicting the mean, highlighting poor predictive performance. In general, higher  $R^2$  values suggest better model performance, as more of the variation in the data is accounted for by the model. However, it is important to note that  $R^2$  alone does not provide information about the actual size or distribution of prediction errors, and it can sometimes be misleading—especially when applied to models evaluated on datasets that differ significantly from the training data. Therefore,  $R^2$  is most effective when interpreted alongside other metrics such as RMSE and MAE, which help quantify prediction accuracy and offer a more balanced assessment of model performance.

### 3.4.2 SUS Test

The System Usability Scale (SUS) is a simple and widely adopted tool for evaluating the usability of interactive systems, such as software applications, websites, or devices. Originally developed by John Brooke in 1986, SUS is designed to provide a quick, reliable, and technology-independent measure of overall usability from the perspective of end users.

It consists of a standardized questionnaire with ten items (see Table 1), each rated on a five-point Likert scale ranging from "Strongly disagree" (1) to "Strongly agree" (5). The questions alternate between positive and negative statements about the system's ease of use, complexity, consistency, and learnability.

After the survey is completed, the individual item scores are converted to a composite score ranging from 0 to 100—but it is not a percentage. The scoring procedure involves subtracting 1 from the user response for odd-numbered items and subtracting the response from 5 for even-numbered items; the sum of these adjusted values is then multiplied by 2.5 to produce the final SUS score.

In terms of interpretation, SUS scores are generally evaluated using benchmark scales: a score of 68 is considered average usability, scores above 80 indicate excellent usability, while scores below 50 suggest poor usability. Because it is easy to administer and can be applied to virtually any system, SUS has become one of the most commonly used usability assessment methods in both research and industry contexts.

| No. | Question                                                                                   |
|-----|--------------------------------------------------------------------------------------------|
| 1   | I think that I would like to use this system frequently.                                   |
| 2   | I found the system unnecessarily complex.                                                  |
| 3   | I thought the system was easy to use.                                                      |
| 4   | I think that I would need the support of a technical person to be able to use this system. |
| 5   | I found the various functions in this system were well integrated.                         |
| 6   | I thought there was too much inconsistency in this system.                                 |
| 7   | I would imagine that most people would learn to use this system very quickly.              |
| 8   | I found the system very cumbersome to use.                                                 |
| 9   | I felt very confident using the system.                                                    |
| 10  | I needed to learn a lot of things before I could get going with this system.               |

Table 1: System Usability Scale (SUS) Questionnaire Items

## 4 ANALYSIS & DESIGN

This section provides a comprehensive overview of the system's functional and non-functional specifications, architectural flow, and interaction patterns. It begins by outlining the functional requirements that define what the system is expected to do, followed by non-functional requirements that specify the desired quality attributes such as usability, reliability, and performance. Next, the section presents two key sequence diagrams to visualize the interaction flows between the user interface, application logic, and backend services during station recommendation and reservation processes. These diagrams help illustrate how data moves through the system, how external services are integrated, and how key decisions are made at runtime.

### 4.1 Functional Requirement

In system design, functional requirements define the essential behaviors, operations, and services that a system must perform to achieve its intended objectives. These requirements describe what the system is expected to do, specifying how it should process inputs, generate outputs, and facilitate interactions between the user and the system. They form the foundation for the system's expected functionality and ensure that the solution meets both technical and user needs. Functional requirements are typically derived from use cases and user stories, helping developers understand and implement the necessary components of the system.

The following functional requirements outline the core capabilities and key processes of the developed system, serving as a blueprint for its intended behavior and performance.

- **The system must allow new users to create an account using a valid email and password and also provide a password recovery option, allowing users to request a password reset email if they forget their password.**

User account management functionality ensures that users can register and manage access to their accounts. The application validates input fields such as email format and required values before making requests to Firebase Authentication. The processes of account creation, login, and password reset email delivery are performed by Firebase Authentication. The application captures the responses and error messages from Firebase and presents appropriate feedback to guide the user through registration, login, and password recovery.

- **The system must allow existing users to log in using their email and password and must maintain the user's authenticated state between app launches, so users remain logged in unless they explicitly log out.**

The login process allows users to access their accounts by entering a valid email and password. The application validates user input and then requests Firebase Authentication to verify the provided credentials. Once authenticated, the user's login state is tracked using local storage (via @AppStorage), enabling the app to remember the user's session across app restarts. This ensures that users do not need to log in again each time they open the app unless they choose to log out. The app provides clear feedback for both successful logins and any errors encountered during the process.

- **The system must present the core features of the app through a tab-based main navigation interface, which includes Map, Explore, and Profile sections. The system must also provide an accessible side menu from the main screen, allowing users to log out of their account.**

The application navigation allows users to easily move between the main sections of the app using a tab bar interface. The tabs provide quick access to the Map, Explore (Best SlotView), and Profile (User Reservations View) screens. In addition, a side menu can be opened from the main screen via a toolbar button. The side menu currently provides a Logout option, allowing users to securely end their session and return to the welcome screen.

- **The system must enable users to explore nearby EV charging stations on a map based on their current location. The system must allow users to search for stations by text, view detailed information about a selected station, and display a driving route for navigation to the chosen station.**

The map interface helps users discover EV charging stations sourced from the project dataset. The system fetches and displays these stations relative to the user's current location. Users can filter the displayed stations by entering text in the search box; to match a station, the full station ID must be entered. When a user taps a station marker on the map, the system retrieves and presents detailed station information, including current availability. If the user chooses to navigate, the system generates and displays a driving route from the user's current location to the selected station using Google Directions.

- **The system must allow users to receive optimized charging slot recommendations based on their EV data, current battery level, and preferred battery threshold at arrival. The system must compute station fit scores considering ETA, predicted availability, and current reservations, and present the ranked list of recommended stations.**

The smart search feature enables users to plan charging stops effectively. Users provide their current EV data (car model, battery level) and set a minimum battery at arrival.

The system calculates the reachable area and fetches nearby stations from the project dataset. It then computes the ETA to each station and predicts availability for the user's estimated arrival time. Current active reservations are considered to adjust the availability. Based on this data, the system computes a Fit Score for each station, ranks the stations accordingly, and displays them to the user. Users can navigate to a selected station or initiate a reservation.

- **The system must allow users to create reservations for selected charging stations and time slots. The system must validate that no overlapping reservation exists and that availability meets a minimum threshold. Users must also be able to view and cancel their active reservations.**

The reservation feature allows users to reserve charging slots in advance. When reserving a slot, the system checks for any overlapping existing reservations for the user and verifies that the selected station has sufficient availability (minimum 50%) for the entire reservation duration. Valid reservations are stored in Firestore. Users can view their active reservations in the profile section and may cancel reservations if needed. The system ensures that users cannot double-book or reserve unavailable time blocks.

## 4.2 Non-Functional Requirements

Non-functional requirements define the quality characteristics and performance expectations that shape how the system operates, rather than specifying its core functions. These requirements are essential for delivering a stable, efficient, and user-friendly experience. They cover key attributes such as response time, availability, usability, reliability, scalability, and security.

By establishing how the system should behave under different conditions, non-functional requirements help ensure consistent performance and guide long-term system maintenance. The following non-functional requirements describe the quality standards and operational constraints expected from the developed system to meet both technical goals and user needs.

- **The system must present clear and simple UI flows for new users (Welcome screen, Sign Up) and existing users (Log In, Forgot Password). Error messages must be shown when user actions are invalid.**

The account management flow provides users with a straightforward experience when registering, logging in, or recovering their account. The Welcome screen clearly separates the Sign Up and Log In paths. Input validation is performed quickly, and if users provide invalid data, descriptive error messages are displayed to help them correct their input.

The map screen allows users to explore nearby EV charging stations through a familiar map interface. Stations are displayed as interactive markers that users can tap to view detailed information in a bottom sheet. In this view, a clearly visible navigation button is provided, allowing users to easily see and access the navigation route to the selected station.

- **Users must be able to easily adjust car data and battery preferences. The ranked station list must clearly present ETA and availability in a user-friendly format.**

In the BestSlotView screen, users can update their car model and battery level through simple UI controls. The recommended stations list presents ETA and availability in a clear and readable format, enabling users to make informed decisions.

- **The reservation process must provide clear feedback (success, errors). Users must be able to easily view and manage their reservations.**

The reservation flow displays clear messages when a reservation is successful or if an error occurs (e.g., availability too low). In the profile screen, users can view their active reservations and cancel them if desired.

- **The system must securely store and manage user credentials through Firebase Authentication.**

User authentication is handled through Firebase Authentication, which ensures that user credentials are securely stored and processed. The system delegates encryption and identity management to Firebase. The app does not store or handle raw passwords directly, and all sensitive authentication operations are performed via Firebase's secure APIs.

- **The login state must be persisted efficiently using lightweight storage (@AppStorage). The main navigation must load quickly after login.**

The system uses @AppStorage to maintain the user's login state efficiently across app restarts. After a successful login, the main navigation (Tab Bar) is displayed immediately to provide a smooth user experience.

- **The system must integrate with Google Maps SDK to display maps and markers, and with Google Directions API to fetch and display driving routes. Google Directions API must be used for ETA computation.**

The map functionality uses Google Maps SDK to present interactive map views and station markers. For navigation and ETA computation, the system calls the Google Directions API to provide accurate driving routes and estimated arrival times.

- The system must compute and present the initial ranked list of stations within a reasonable response time after the user taps "Search Best Slots." The system must allow users to progressively load more recommendations.

After the user initiates a smart search, the system calculates reachable stations, ETA, availability, and fit scores based on the dataset used in the project and test location. The ranked list of stations is presented in a timely manner, and additional recommendations can be loaded progressively upon user request.

### 4.3 Sequence Diagrams

This section presents two sequence diagrams illustrating core flows in the project. The first diagram shows how the system generates recommended stations by retrieving station data, calculating ETA, and predicting availability. The second diagram depicts the reservation process, including conflict checks, availability validation, and storing the reservations.

#### 4.3.1 Recommended Stations Sequence Diagram

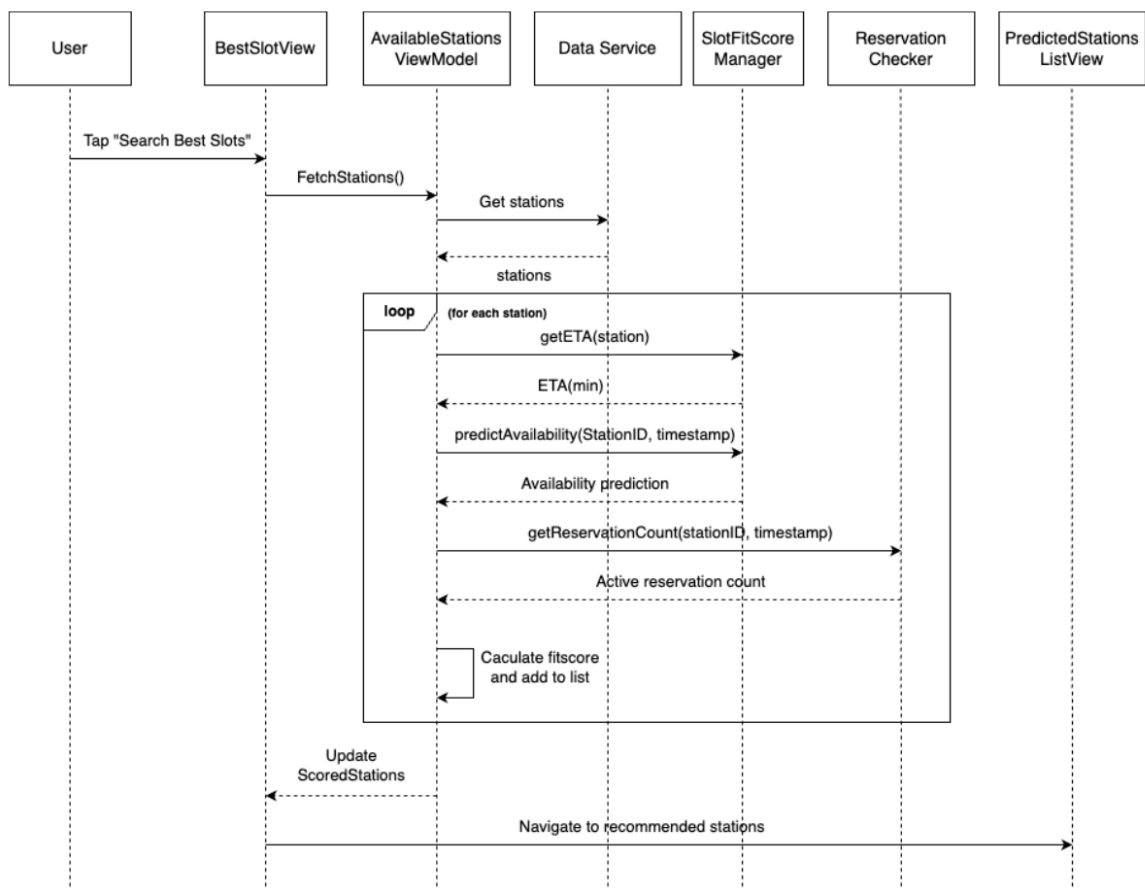


Figure 4.1: Recommended Stations Sequence Diagram

Figure 4.1 illustrates the sequence diagram that outlines the runtime flow of the recommended station display functionality when a user initiates a search for optimal charging slots. The process begins on the *BestSlotView* screen, where the user taps the "Search Best Slots" button. This action triggers a call to the *fetchInitialStations()* method within the *AvailableStationsViewModel*. This method is responsible for coordinating the necessary data retrieval and computations to generate and return an optimized list of nearby charging stations to the user.

The first step performed by *AvailableStationsViewModel* is to calculate the user's remaining driving range based on their current battery level and the selected battery threshold for arrival. Using this calculated range, the system calls *UserSessionManager.fetchNearbyStations()* to query nearby charging stations from Firestore within the drivable radius. In the sequence diagram, this interaction is abstracted as "Get stations" and the response as "stations". After fetching the nearby stations, the system performs a pre-sorting phase based on ETA. For each station, the app requests an ETA from *SlotFitScoreManager.getETA()*, which internally uses the Google Directions API to compute the driving time from the user's current location to each charging station. Although this pre-sorting phase happens prior to the main loop, the diagram presents the ETA request inside the loop for conceptual simplicity. Following ETA estimation, the system enters its main processing loop, iterating over each station to compute its Slot Fit Score. For every station, the following operations are performed in sequence:

- **Availability Prediction:** The app calls *SlotFitScoreManager.predictAvailability()*, which prepares a feature payload for the target station and timestamp, sends it via socket to a hosted machine learning model server, and receives an availability prediction for that time slot.
- **Reservation Check:** The system calls *ReservationChecker.getReservationCount()*, which queries the Firestore reservations collection to determine how many active reservations overlap with the selected time block for that station.

With both the predicted availability and active reservation count in hand, the system calculates an adjusted fit score. This score reflects the station's likelihood of having available charging plugs when the user arrives, taking into account both predicted demand and currently reserved slots. Each computed *ScoredStation* object (including ETA, fit score, and prediction time) is added to the *scoredStations* list within the view model.

Once the initial batch of stations is processed, the updated *scoredStations* list triggers a state change in *BestSlotView*. This results in a navigation transition to the *PredictedStation-ListView*, where the recommended charging stations are displayed in a ranked list, sorted by fit score and ETA. The user can then explore navigation options or make reservations through this interface.

### 4.3.2 Reservation Sequence Diagram

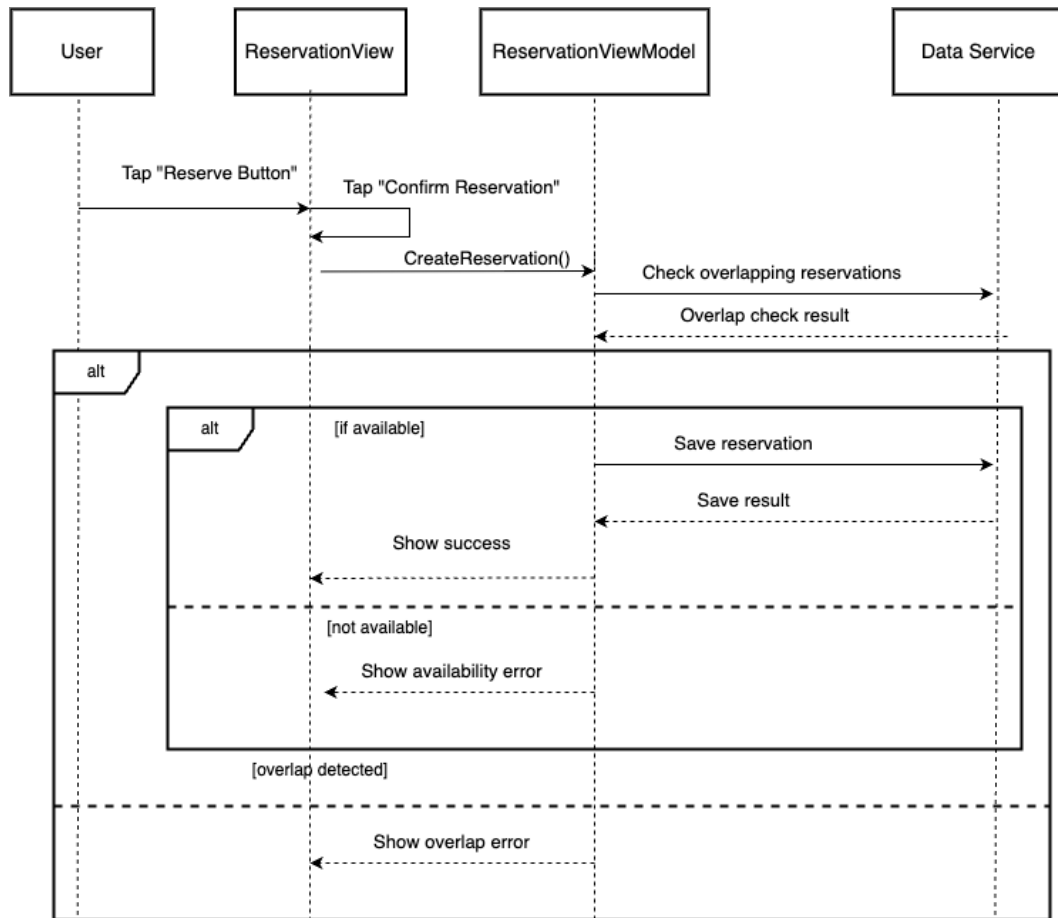


Figure 4.2: Rezervation Process Sequence Diagram

Figure 4.2 illustrates the sequence diagram that describes the runtime flow for the reservation creation process within the system. This process is designed to ensure that users can only create valid and conflict-free reservations that align with real-time station availability constraints. The flow begins when the user taps the “Confirm Reservation” button on the ReservationView screen, which initiates the createReservation() function in the ReservationViewModel component. The reservation process begins by checking for any existing active reservations that overlap with the requested time window. This is done using the checkForOverlappingReservations() method, which queries the Firestore database. If an overlap is found, the system follows the [overlap detected] path and displays an error to the user, preventing duplicate bookings and ensuring consistency. If no conflicts are detected, the system validates the predicted availability for the selected 15-minute time blocks. A minimum threshold of 50% availability is required to proceed. If any block falls below this threshold, an availability error is shown to the user, prompting them to adjust their reservation. If all checks pass, the reservation is saved and confirmed.

## 5 IMPLEMENTATION

This section presents the component-by-component implementation of the proposed system Figure 5.1, Smart EV Charging Slot Reservation with Dynamic Slot Suggestion.

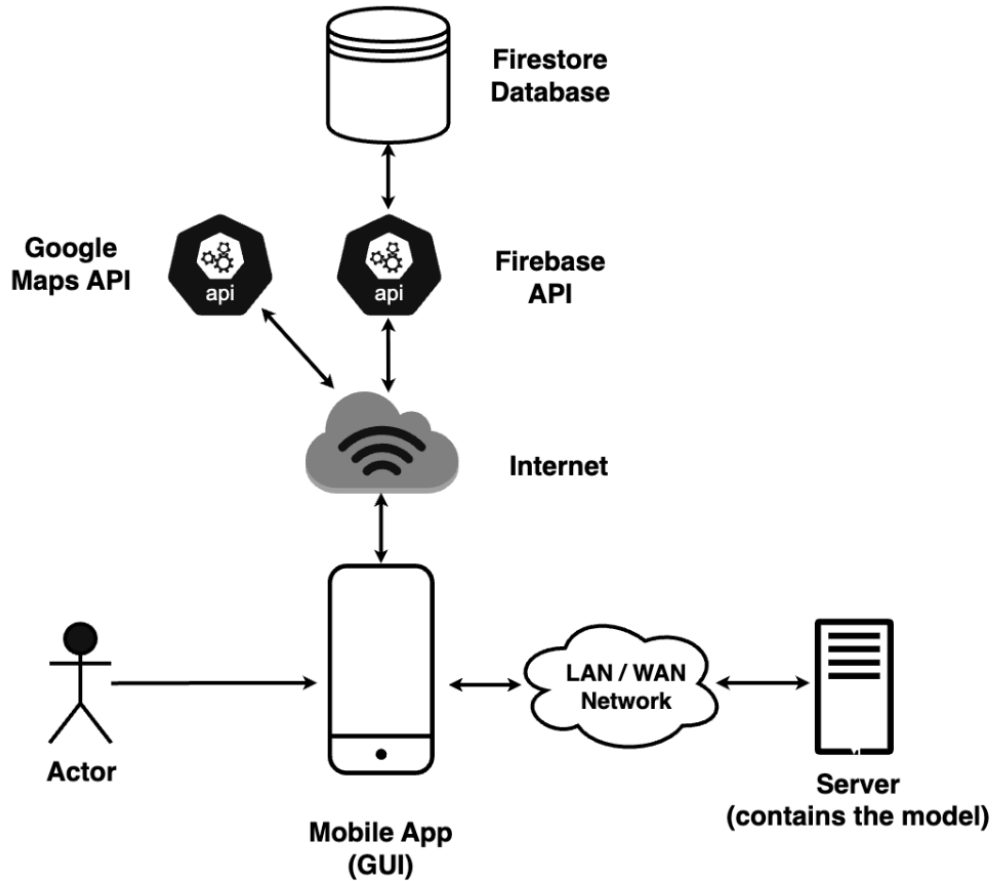


Figure 5.1: Overall Architecture of the System

### 5.1 Availability Prediction Model

This subsection provides an overview of the dataset employed for model training, along with the feature extraction process and the training approach.

#### 5.1.1 Dataset

During the development of this project, the dataset provided as part of the Codalab Smart Mobility Data Challenge [1] (conducted between October 3 and December 5, 2022) was used. The dataset consists of 15-minute interval records collected from 91 charging stations in Paris between July 3, 2020, and February 18, 2021. It is a comprehensive dataset containing a total

of 1,851,083 rows and 13 columns. As illustrated in Figure 5.2, each record includes four key columns that represent the status of each charging station:

- **Available:** The number of charging plugs that are free and available
- **Charging:** The number of plugs currently in use for active charging
- **Passive:** The number of plugs connected to a vehicle but not actively charging (idle)
- **Other:** The number of plugs that are offline or out of order

The sum of these four columns is always 3, as each charging station is equipped with exactly three plugs. The remaining columns provide additional contextual information, including timestamp, station name, geographical location, and area.

Additionally:

- **tod (time of day):** Indicates which 15-minute time interval the record belongs to within the day
- **dow (day of week):** Indicates which day of the week the record corresponds to

These time-based variables (*tod* and *dow*) and station-specific variables were later used as input features in the machine learning model to predict future station availability.

|   | date       |          |     |       | Station           | Available | Charging | Passive | \ |
|---|------------|----------|-----|-------|-------------------|-----------|----------|---------|---|
| 0 | 2020-07-03 | 00:00:00 |     |       | FR*V75*EBELI*1*1  | 3         | 0        | 0       |   |
| 1 | 2020-07-03 | 00:00:00 |     |       | FR*V75*EBELI*10*1 | 2         | 1        | 0       |   |
| 2 | 2020-07-03 | 00:00:00 |     |       | FR*V75*EBELI*11*1 | 2         | 1        | 0       |   |
| 3 | 2020-07-03 | 00:00:00 |     |       | FR*V75*EBELI*12*1 | 3         | 0        | 0       |   |
| 4 | 2020-07-03 | 00:00:00 |     |       | FR*V75*EBELI*13*1 | 3         | 0        | 0       |   |
|   | Other      | tod      | dow | trend | Latitude          | Longitude | Postcode | area    |   |
| 0 | 0          | 0        | 6   | 27    | 48.855667         | 2.354089  | 75004    | south   |   |
| 1 | 0          | 0        | 6   | 27    | 48.864240         | 2.397724  | 75020    | east    |   |
| 2 | 0          | 0        | 6   | 27    | 48.861016         | 2.283080  | 75016    | west    |   |
| 3 | 0          | 0        | 6   | 27    | 48.824388         | 2.336857  | 75014    | south   |   |
| 4 | 0          | 0        | 6   | 27    | 48.882640         | 2.335728  | 75009    | north   |   |

Figure 5.2: First five row from Dataset

### 5.1.2 Feature Extraction and Model Training

The model aims to predict the availability rate of a charging station at a given time using historical data. To respect the time series nature of the dataset, a chronological split was used: data from July 3 to December 10, 2020 was used for training, December 11, 2020 to January 13, 2021 for validation, and January 14 to February 18, 2021 for testing. This corresponds to an approximate 70/15/15 split, enabling realistic future forecasting. In addition, six new

features were engineered from the original dataset. The purpose of these features is to enhance the predictive accuracy of the model.

**station\_dow\_avg\_available** is a statistical attribute that represents the average number of available charging plugs for each station on a specific day of the week. This feature is designed to capture the general weekly usage patterns of each station. To compute it, the training dataset was grouped by each station and day-of-week pair, and the average available plugs was calculated for each group. For example, this feature simply reflects the overall average availability on Mondays for that station. This information helps the model learn day-specific demand variations, thereby improving the accuracy of future availability predictions.

**area\_avg\_available** is a statistical attribute computed based on the average number of available charging plugs across all stations located within the same geographical area (such as a neighborhood, district, or postal code). It is calculated by taking the average of *Available* values for all stations within each *area* using the training dataset. This feature enables the model to be sensitive not only to individual station trends, but also to the broader occupancy patterns of the surrounding area. As a result, even if there is a temporary lack of data for a specific station, the model can leverage the area-level availability trend to provide contextual support and improve prediction accuracy. This is especially beneficial in high-density areas where multiple stations exist, as the *area\_avg\_available* feature helps the model learn location-specific demand variations.

**station\_smoothed\_trend** reflects the temporal occupancy trend of each station, computed based on historical data. It was generated using an Exponential Weighted Average (EWA) technique, which produces a smoothed trend by assigning greater weight to more recent data points and less weight to older ones. During the calculation, only previous days' data was used, the current day's data was excluded, to avoid data leakage and ensure a proper time-aware feature. The smoothing was designed to be sensitive to short-term fluctuations, enabling the model to better capture increasing or decreasing occupancy patterns over time. This feature is particularly helpful in distinguishing temporal dynamics, such as differing usage patterns between weekdays and weekends, thereby improving the model's ability to learn time-dependent trends and enhance overall prediction accuracy.

**previous\_available** represents the number of available charging plugs at the previous time step (i.e., in the preceding 15-minute interval) for each station. This feature leverages the time series nature of the data to help the model learn how the current state of a station has evolved over time. It was calculated by sorting the observations for each station in chronological order and assigning the *Available* value from the previous row as the *previous\_available* feature. In this way, the model is provided with temporal context regarding the station's occupancy dynamics, allowing it to better capture patterns such as increasing congestion or decreasing

occupancy. This feature introduces temporal continuity into the model and plays an important role in helping the model understand and respond to sudden changes in station usage.

**previous\_charging** is similar to *previous\_available* and represents the number of actively charging vehicles at the previous time step (i.e., in the preceding 15-minute interval) for each station. This time series-based feature was designed to help the model learn the short-term usage trends of each station. It was computed by sorting the data chronologically for each station and shifting the *Charging* values so that the previous value was assigned to each row. In doing so, the model gains additional temporal context regarding recent charging activity, which can improve its ability to predict near-future station availability.

In this study, the LightGBM (Light Gradient Boosting Machine) algorithm was used to predict the availability rates of electric vehicle charging stations for specific days and time intervals. LightGBM is a tree-based boosting algorithm that differs from traditional gradient boosting methods by employing a histogram-based splitting technique and a leaf-wise tree growth strategy. These features enable the model to achieve high accuracy, faster training times, and better memory efficiency, making it a powerful and scalable solution for modern regression tasks.

During model training, a total of nine features were used: station identifier, day and time information (*Station*, *dow*, *tod*), historical statistical features (*station\_15min\_avg\_available*, *station\_dow\_avg\_available*, *area\_avg\_available*), time series trend (*station\_smoothed\_trend*), and the occupancy status from the previous time step (*previous\_available*, *previous\_charging*). The target variable was defined as the normalized number of available charging sockets, represented by availability rate. The model was trained with a learning rate of 10% and configured with 100 trees. The features *Station*, *tod*, and *dow* were explicitly treated as categorical variables within the model. This setup enables the model to effectively capture time and location-based charging demand patterns while helping to reduce the risk of overfitting.

To support the project development and testing, a simple local TCP socket server was implemented to facilitate communication between the mobile application and the predictive model. The mobile app sends prediction requests in JSON format to the server, which processes the input and returns a prediction. This local server setup allows for rapid testing of the model integration within the app during development, without the need for a full production API or cloud deployment.

## 5.2 Mobile Application & GUI

The application was developed using the MVVM (Model-View-ViewModel) architecture. MVVM is a software design pattern that is widely used in mobile application development. This architecture separates the application into three distinct layers: Model (Data Layer),

View (User Interface), and ViewModel (Presentation Logic). The Model layer represents the data and business rules of the application. The View layer is responsible for rendering the user interface but does not contain any business logic or data processing. The ViewModel acts as a bridge between the View and the Model; it prepares the data, applies business logic, and formats it for display in the UI. MVVM helps reduce dependencies between layers, providing advantages such as modularity, testability, ease of maintenance, and reusability.

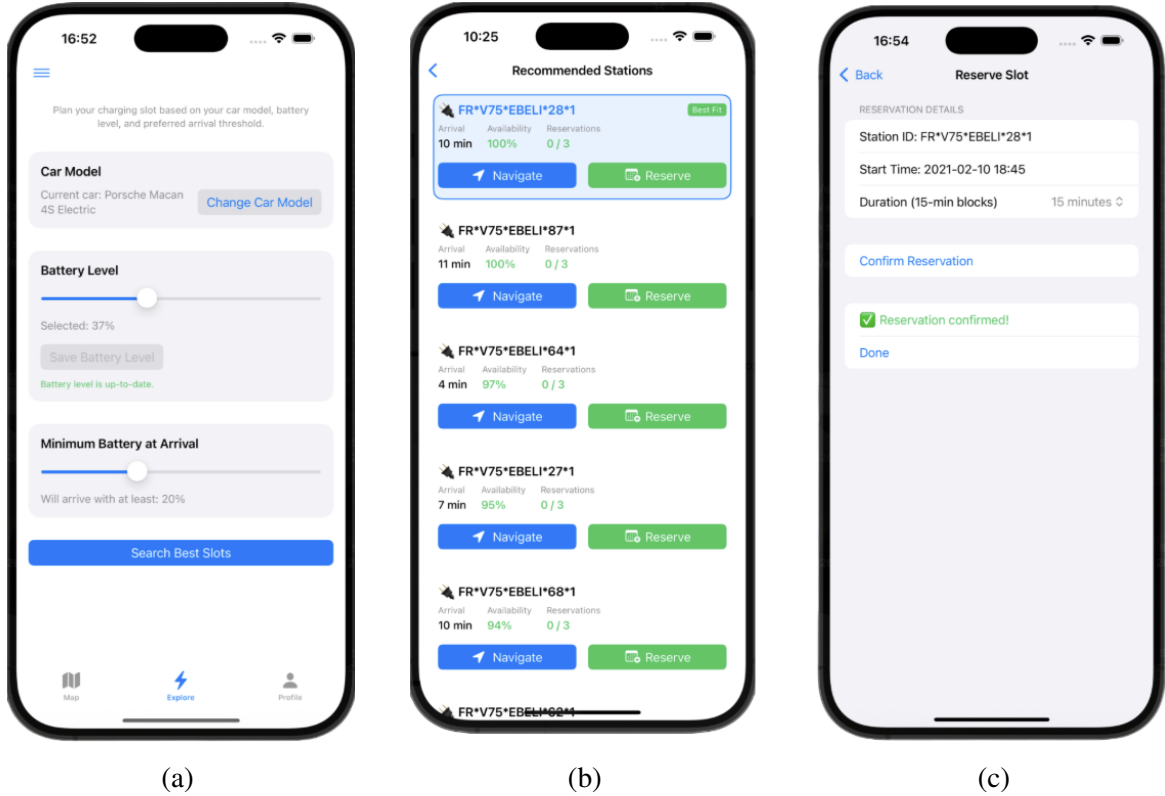


Figure 5.3: Explore tab screens.

**BestSlotView** serves as the main entry screen for users to plan their EV charging session based on their vehicle's battery state and preferences. On this screen, the user can select their car model, adjust their current battery level, and set a desired minimum battery level for arrival (battery threshold) see Figure 5.3a. When the user taps the "Search Best Slots" button, the screen triggers a call to the *AvailableStationsViewModel*, which handles the logic for fetching and ranking nearby charging stations. It calculates reachable distance based on the vehicle's range, retrieves nearby stations from Firestore via the shared *UserSessionManager*, determines predicted ETA using Google Maps Directions API, and predicts station availability through *SlotFitScoreManager*, which integrates station status data and a trained availability model accessed via a socket-based API. Reservation data is also factored in to reflect realistic availability. The resulting list of *ScoredStation* objects, each containing a station ID, ETA, and fit score, is published back to the view. Once available, the app navigates to

*PredictedStationListView*, which presents this ranked list of stations to the user Figure 5.3b. On this screen, users can interact by choosing to navigate to a station or make a reservation, with additional functionalities such as pull-to-refresh and progressive loading of more stations. Throughout this flow, the app adheres to the MVVM architecture, where the views focus solely on presenting the UI and responding to user interactions, while *AvailableStationsViewModel* encapsulates all data processing, business logic, and state management in the background.

**PredictedStationListView** displays the personalized list of charging stations recommended for the user Figure 5.3b. The screen presents key station details, such as estimated arrival time and an availability-based fit score, enabling users to make informed decisions. It also allows users to interact with the recommendations by refreshing the list, loading additional stations, initiating navigation, or proceeding to make a reservation Figure 5.3c. Consistent with the MVVM pattern, the view observes state updates published by *AvailableStationsViewModel* and focuses purely on the presentation and user interactions, while the view model handles the logic required to maintain and update the recommendation data.

### 5.2.1 Swift & SwiftUI Library

In this project, SwiftUI is used as the main technology to create the entire user interface of the mobile app. SwiftUI allows developers to build screens in a clear and structured way, where the app's screens automatically update when the data changes. The app's main screens are built with SwiftUI components such as *NavigationStack* (which allows moving between screens), *ScrollView*, *VStack*, *HStack*, and *List* (which help arrange content vertically, horizontally, or as a list). User inputs are handled with components like *Button* (for actions), *Slider* (for selecting values like battery level), *Picker* (for choosing a car model), and *ProgressView* (to show loading states).

A key feature of SwiftUI is that it uses reactive state management. This means that when important data in the app changes, the corresponding screen elements are automatically refreshed. In this project, reactive state management is implemented using property wrappers like *@State*, *@ObservedObject*, and *@StateObject*. For example, *@State* is used to hold simple values inside a screen (such as the currently selected battery level), while *@ObservedObject* and *@StateObject* are used to observe external data sources (such as the list of available stations or the user's car information). When these observed objects update their data (for example, when new station predictions are fetched), SwiftUI automatically triggers a re-render of the screen, so the user always sees the latest information. This approach makes the app more efficient, interactive, and easier to maintain.

The project uses Swift's features such as object-oriented and protocol-oriented program-

ming to organize its logic and data structures. Important logic is written in classes like *AvailableStationsViewModel* and *ReservationViewModel*, which store app data (state) and perform operations like calculating station scores or creating reservations. The app also defines lightweight data models using structs, such as *ChargingStation*, *ScoredStation*, and *Reservation*. These structs adopt protocols like *Identifiable* (which gives each item an id so it can appear in lists) and *Codable* (which allows the data to be easily saved or loaded), making data handling safe and efficient.

To perform background operations, the project uses completion handlers (also called closures) and *DispatchGroup*. These allow multiple tasks such as calculating ETA, fetching predictions, or checking reservations to run in the background without freezing the user interface. The project also makes extensive use of Swift's built-in types such as *Date*, *Calendar*, and *DateFormatter* to handle time-related calculations, such as computing when a reservation ends or formatting dates for display. The *CLLocation* and *CLLocationCoordinate2D* types are used for location-based calculations, such as measuring the distance between the user and charging stations.

Swift's type safety and optional handling help ensure that missing or incorrect data does not cause app crashes. The project uses guard statements to make sure required values (like battery level or car model) are available before performing calculations. Additionally, error handling and safe unwrapping of optionals are used throughout the code to manage unexpected situations, so the app remains stable even when data is missing or delayed.

### 5.3 API & Database

The project integrates both Google APIs and Firebase APIs to enable key functionalities of the application. The Google Maps Directions API is used to calculate driving routes and ETA between the user's current location and nearby charging stations. These ETAs are used during the station scoring process to recommend optimal stations based on proximity and expected arrival time. The app also uses the Google Maps SDK (Software Development Kit) to visually display driving routes on the map when the user selects a station for navigation.

For backend services, the project uses several Firebase APIs. Firebase Authentication is employed to handle user account management, including login, logout, and password reset. The app also uses Firebase Firestore as its cloud database to store and manage important data such as user EV profiles, charging station metadata, real-time station availability, and reservation records. The app interacts with Firestore dynamically: it reads and writes data asynchronously, receives real-time updates to user EV information, retrieves station availability features to support the machine learning-based availability prediction, and performs checks for overlapping reservations during the booking process. Together, these Google

and Firebase services provide seamless support for real-time navigation, data-driven station recommendations, and secure cloud-based data management within the app.

In Firestore, four main collections have been defined specifically for the application. The **users collection** contains one document for each user, storing information such as the user’s email address, vehicle model, battery capacity, average consumption rate, and current battery level. After the user logs in, this information is created or updated by the *UserSessionManager* ViewModel.

The **car\_models collection** (see Figure 5.4) stores the list of electric vehicles that users can manually select within the app. Each document in this collection represents a specific vehicle and contains properties such as name, avgConsumptionRate, and batteryCapacity.

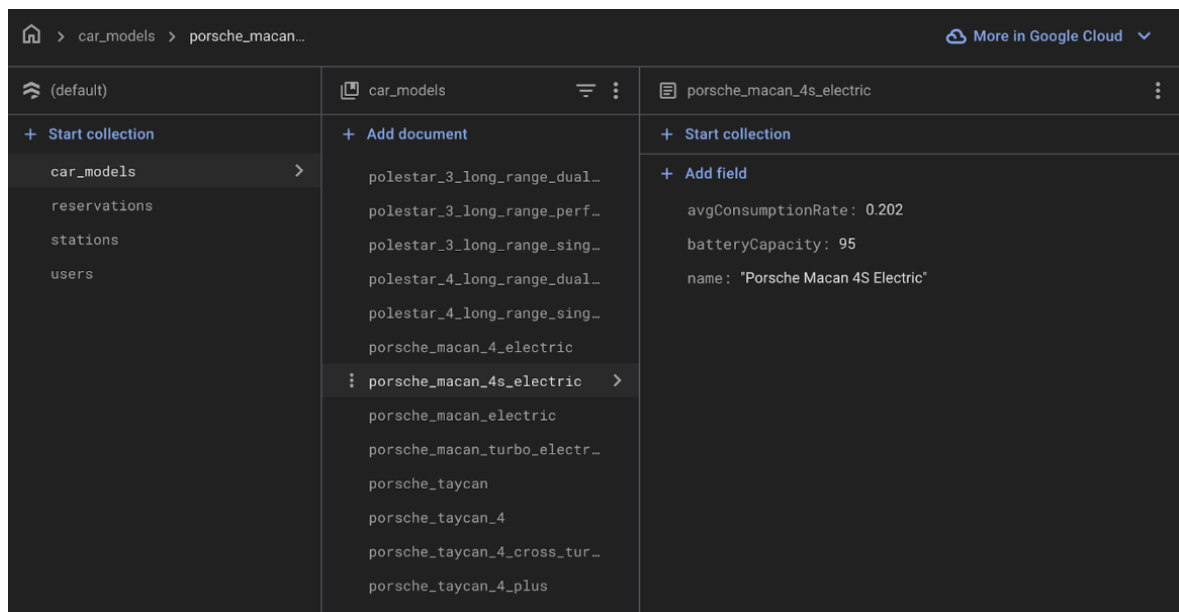


Figure 5.4: Car models collection

The **stations collection** contains the basic data for all charging stations (see Figure 5.5). Each document corresponds to a station and includes fields such as station\_id, latitude, longitude, postcode, area, and total\_plugs (see Figure 5.8). Additionally, each document contains a nested status collection. All data in the Stations collection was derived from the test set of the dataset used in this project.

The **status sub-collection** (see Figure 5.6) is defined for each station and contains the corresponding columns from the test set for specific date-time intervals. Each document represents a unique time slot (for example, “2021-02-10 10:30”). The fields area\_avg\_available, previous\_charging, station\_15min\_avg\_available, station\_dow\_avg\_available, and station\_smoothed\_trend are directly used as essential input features for the prediction model.

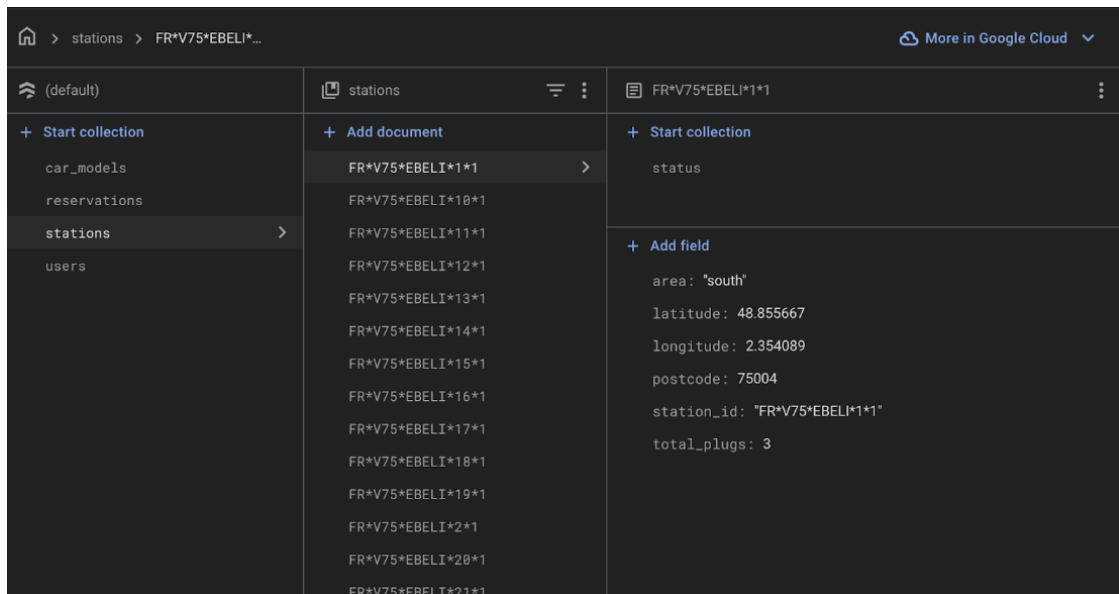


Figure 5.5: Stations collection

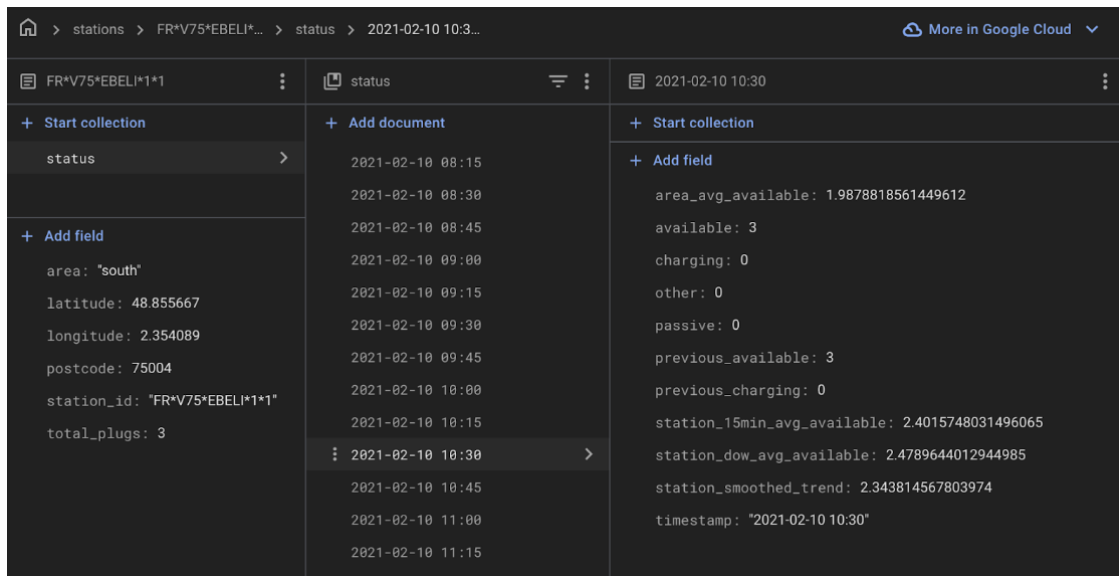


Figure 5.6: Stations status sub-collection

Finally the **reservations collection** stores the reservations made by users. Each document represents a reservation where a user books a station for a specific start time and a specified number of time blocks. These documents are created by the *ReservationViewModel* and can be updated with status information (active/cancelled). In addition, reservation conflict checks and available capacity checks are also performed through this collection. This Firestore structure provides a modular and non-relational, yet logically connected data model within the application. ViewModels such as *UserSessionManager*, *AvailableStationsViewModel*, and *ReservationViewModel* interact continuously with Firestore to manage and update data in real time.

## 6 TEST & RESULTS

This section presents a comprehensive evaluation of both the prediction model and the application itself. First, the model’s performance was assessed using standard regression metrics (RMSE, MAE, and  $R^2$ ) to evaluate its ability to accurately forecast station availability. Next, the application’s core reservation functionality was tested using a scenario-based approach to examine how well it handles real-time user inputs and updates availability in response to active reservations. Finally, the overall usability of the application was evaluated through two methods: the System Usability Scale (SUS), which provides a standardized assessment of system usability, and a supplementary user questionnaire. This additional questionnaire explored specific aspects of the user experience, such as navigation flow, page accessibility, visual clarity, and design aesthetics, offering deeper insight into the app’s interface quality and usability from the user’s perspective.

### 6.1 Model Evaluation

To evaluate the accuracy of the model used in the application, we applied specific performance metrics, including RMSE, MAE, and  $R^2$ , which were previously explained in detail in the Methodology section. These metrics provide different perspectives for assessing the model’s predictive performance and allow us to interpret its overall accuracy.

**The RMSE metric** measures the average deviation of the model’s predictions from the actual values, giving greater weight to larger errors. In other words, a higher RMSE value indicates that the model occasionally produces large prediction errors. In this study, since the target variable is the availability rate, which is normalized between 0 and 1 (representing the ratio of available charging plugs out of a total of three plugs per station), the RMSE is interpreted within this same range. Values closer to 0 indicate more accurate and reliable predictions. In our model, the obtained Validation RMSE is 0.1564, as shown in Table 2, which suggests that the predicted availability rates are generally close to the actual values, with limited deviation from ground truth.

| RMSE   | MAE    | $R^2$ |
|--------|--------|-------|
| 0.1564 | 0.0754 | 0.841 |

Table 2: Regression Model Evaluation Results

**The MAE metric** measures the average deviation of the model’s predictions from the actual values in a direct and equally weighted manner. Unlike RMSE, MAE does not give additional weight to larger errors, making it a clearer and more straightforward indicator of the model’s average prediction accuracy. A lower MAE value, ideally close to 0, is desirable; typically, an MAE below 0.1 is considered sufficient for delivering an acceptable level of accuracy from a

user experience perspective. In our model, the obtained Validation MAE is 0.0754, as shown in Table 2, indicating that the model predicts with an average deviation of approximately 7.5%, which is regarded as an acceptable and good performance level.

**The Coefficient of Determination ( $R^2$ )** is an important metric that shows how well the model explains the changes (variance) in the data. Variance means how much the data values change around their average. For example, EV charging station availability rates can change depending on time of day, day of the week, or how often a particular station is used. In other words, the data may follow certain patterns over time and across locations. The  $R^2$  value is between 0 and 1. A value closer to 1 means that the model has done a good job of learning and explaining these patterns in the data. A high  $R^2$  also means that the model does more than just predict the average, it can also follow more complex trends. In this study, the Validation  $R^2$  is 0.8410 (see Table 2), which means that the model explains about 84% of the total variance in the data. It also shows that the model can accurately predict availability rates depending on time, day, and station. This result suggests that the model is strong, reliable, and can perform well in different usage situations.

When considered together, the RMSE and MAE results suggest that the model produces balanced and reliable predictions, with acceptable average deviations and only occasional larger errors. This level of performance indicates that the model is well-suited for integration into a user-facing EV charging reservation system, where consistency and accuracy are critical for a smooth user experience. Additionally, the high  $R^2$  score shows that the model effectively captures underlying usage patterns across different times and locations, enhancing its ability to support real-time decision-making. While results are strong overall, the gap between RMSE and MAE highlights rare outlier cases, suggesting future improvements could involve incorporating external contextual features such as holidays, weather, or local events to further increase prediction robustness.

## 6.2 Application Reservation Test

To test the reservation feature of the application, a specific test scenario was designed and executed. In this scenario, a day with a relatively high average charging level and complete, reliable data was selected from the dataset. Based on these criteria, February 10, 2021, at 18:30 was chosen as the test time. The user manually entered the required vehicle and battery information through the application interface (Figure 6.1a). Next, the application presented the available stations to the user, ranked according to the predicted availability rates in the system (Figure 6.1b). After the user selected a station from the recommended list, the system automatically calculated the start time of the reservation Figure 6.1c, rounding it to the nearest 15-minute interval. This rounding is essential because the dataset is structured using 15-minute time intervals, meaning that station availability values are only available

at these fixed timestamps. To ensure that predicted availability values can be fairly and accurately compared against actual values, this temporal alignment is required—especially when calculating performance metrics such as MAE, RMSE, and  $R^2$ . Thus, in reservation processes or any test case where estimated time of arrival is used, this rounding method is consistently applied to align user behavior with the resolution of the dataset and preserve the integrity of the evaluation.

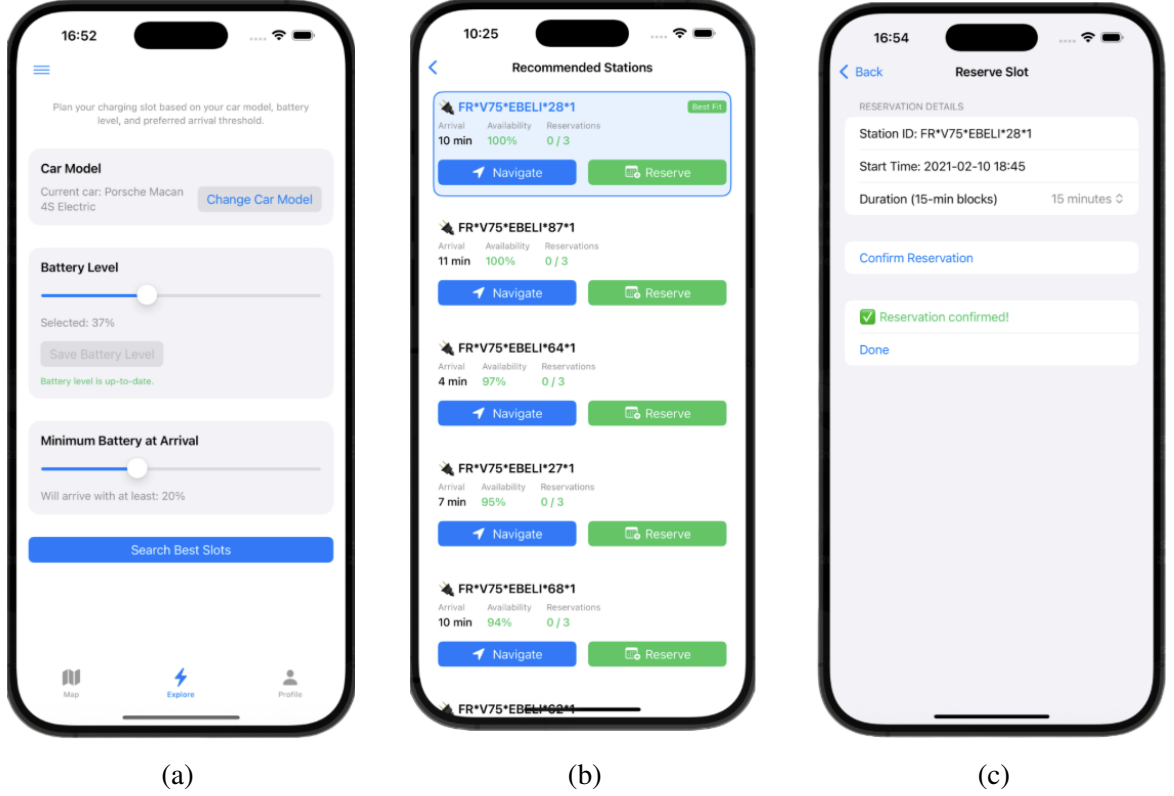


Figure 6.1: Reservation Process

Based on this start time, the user selected the desired reservation duration (expressed as a number of 15-minute blocks) and completed the reservation process. The resulting reservation can be observed in Figure 6.2a. The application’s dynamic reservation update mechanism ensures that users always have access to the most up-to-date and accurate availability information. For example, if a user or a new user reopens the Recommended Stations screen, they may observe that the availability rate for a previously reserved station (Figure 6.2a), such as FRV75EBELI\*28\*1, has been updated from 100% to 66% and the reserved plug count out of the total plug count has also been updated (see Figure 6.2b). This is because each time the Recommended Stations screen is opened, the application recalculates the current availability rates for the relevant stations. During this process, the model first retrieves the predicted availability rate for each station and user-specific estimated arrival time. Then, it queries the Firestore database to check for any active reservations for the same time slot.

The application combines these two pieces of information to compute and display the real-time availability rate for each station to the user. For example, if the model initially predicts 100% availability for station FRV75EBELI\*28\*1 and there is one active reservation for the same time slot, the station's total capacity of 3 plugs is taken into account. Since 1 plug is reserved, the system calculates that 2 plugs remain available, and an updated availability rate of 66% is displayed to the user. Similarly, if the model initially predicts 70% availability, this percentage is first converted into the equivalent number of plugs (e.g., approximately 2.1 plugs for a station with 3 plugs). Then, the number of active reservations is subtracted from this value to determine the updated number of available plugs. Finally, this value is converted back into a percentage availability rate, which is presented to the user. Through this mechanism, users always have access to the most current and nearly accurate availability information, helping them avoid unnecessary reservation attempts for time slots that are already full. By dynamically combining model predictions with real-time active reservation data, the application aims to provide a more reliable and realistic reservation experience for users.

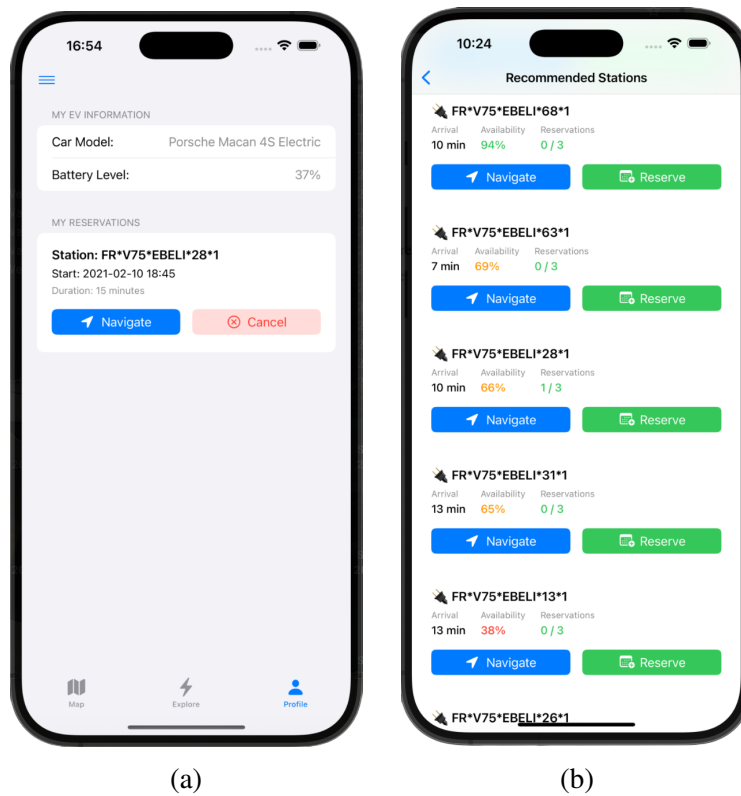


Figure 6.2: Reservations and Recommended Station Update

It is important to note that, once a reservation is made, the system assumes that one charging plug will remain available for the user at the reserved time. From the application's perspective, that plug is treated as taken and removed from the available capacity. The system does not control the physical charging infrastructure or guarantee real-world availability at the station.

Instead, this threshold is designed to improve the overall experience for users who do not make a reservation but instead check availability in real time and go directly to the station. By allowing reservations only when availability is predicted to be above 50%, the system aims to minimize conflicts between users with reservations and users without, helping to ensure fair and efficient use of charging resources. Users attempting to reserve a time slot below this threshold are informed that the reservation is not allowed (see Figure 6.3b).

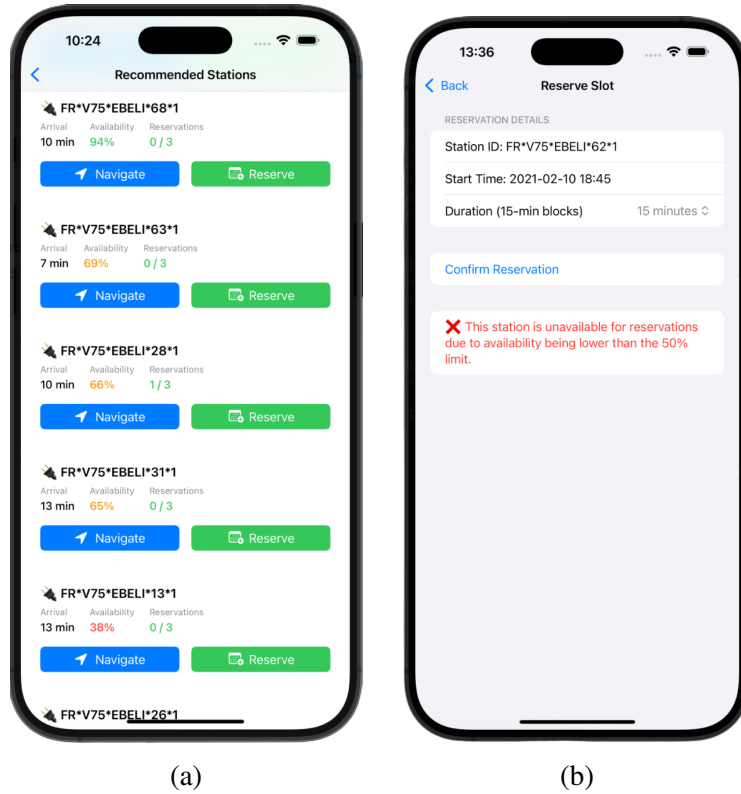


Figure 6.3: Reservation Limit

### 6.3 Application Usability Tests

In this study, the System Usability Scale (SUS) was selected as the primary tool for evaluating the usability of the application. The SUS is a widely recognized and validated method for obtaining a quick, reliable measure of overall system usability, making it well-suited for benchmarking user experience across different types of applications. It offers a standardized way to assess usability that is both efficient and comparable.

In addition to the SUS, a short application-specific questionnaire was prepared to collect more detailed feedback on certain aspects of the app, such as navigation, visual design, and content clarity, which are not fully covered by the standard SUS questions. This additional set of questions provided more targeted insights into interface design and user expectations. The aim of combining these two tools is to gain a clearer understanding of users' experiences and

expectations, identify areas for improvement, and guide future development of the application.

### **6.3.1 SUS Test**

To assess overall usability, the System Usability Scale (SUS) was applied, using the standard 10-item questionnaire, as explained in the Methodology section. Based on responses from 24 participants, the calculated SUS score was 84.06 out of 100, which indicates a level of usability that is considered excellent. According to established SUS benchmarks, a score above 80 typically reflects a highly usable system with strong user satisfaction. This result suggests that the developed application offers a very positive user experience and is well-positioned for practical adoption.

### **6.3.2 Application Questionnaire**

The application-specific questionnaire was developed with the goal of gaining a deeper and more detailed understanding of users' overall experiences and impressions while interacting with the implemented app. The questionnaire was composed of a total of five questions. Four of these were core usability questions, each rated on a Likert scale from 1 to 5, and designed to assess key interface elements such as the clarity of the navigation structure, the ease of accessing different pages, the comprehensibility of content displayed within the app, and the visual quality of the design and interface elements (see Table 3). In addition to these, one background question was included to gather demographic and contextual information, specifically regarding whether participants had prior experience with similar electric vehicle charging applications.

In total, the questionnaire was completed by 24 participants. A significant majority (79.2%) were in the 18–25 age range, which reflects a demographic likely to be technologically active but possibly less experienced with EV charging apps. Furthermore, 62.5% of respondents reported having no prior experience with any similar app, indicating that the feedback gathered is especially valuable in terms of evaluating first-time user impressions.

The overall usability ratings were highly positive. Each of the four main usability questions received an average score above 4.5 out of 5 (see Figure 6.4), demonstrating a strong user consensus regarding the effectiveness of the application's interface and ease of use. Among the highest-rated aspects were ease of page access (Q2, avg. score 4.83) and overall clarity of navigation (Q1, avg. score 4.63), followed by visual design (Q4, 4.58) and content clarity (Q3, 4.54). These scores suggest that the application successfully delivers a clean, functional, and user-centered design that aligns well with expectations.

These quantitative findings were supported by open-ended user feedback. Multiple participants described the app as “very easy to use,” “straightforward,” and “very helpful.” One user

remarked, “I wouldn’t recommend anything else specifically,” indicating high satisfaction. Another user commented, “All elements in the application are as they should be: clean, easy to understand, and functional.” A different respondent highlighted the app’s practicality by saying, “I found it very useful and practical, and I wouldn’t want any changes or additions.”

At the same time, a few constructive suggestions were offered for future improvement. One user noted that “a visual explanation on the interface would help users understand things faster and more easily.” Another suggested the inclusion of details about amenities available at charging stations, such as food, coffee, prayer rooms, or car wash services. Additionally, the ability to view past reservations was mentioned as a desirable feature in later versions. Lastly, while most users were satisfied with the app’s focus, one participant commented that “it is definitely a goal-oriented app, but the aesthetics could be improved,” highlighting a potential area for enhancing visual design in future iterations.

Overall, the results of the questionnaire confirm that the application delivers a strong and user-friendly experience, particularly from the perspective of new users. While the core functionality and interface are widely appreciated, the feedback also provides valuable insights into how the system can evolve with small enhancements to further improve usability and visual appeal.

| Question | Question Text                                                              |
|----------|----------------------------------------------------------------------------|
| Q1       | The navigation within the app was clear and easy to use.                   |
| Q2       | I was able to access the page I wanted quickly and easily.                 |
| Q3       | The information on the pages was clear and sufficient.                     |
| Q4       | The app’s colors and interface elements were clear and visually appealing. |

Table 3: Usability Test Questions

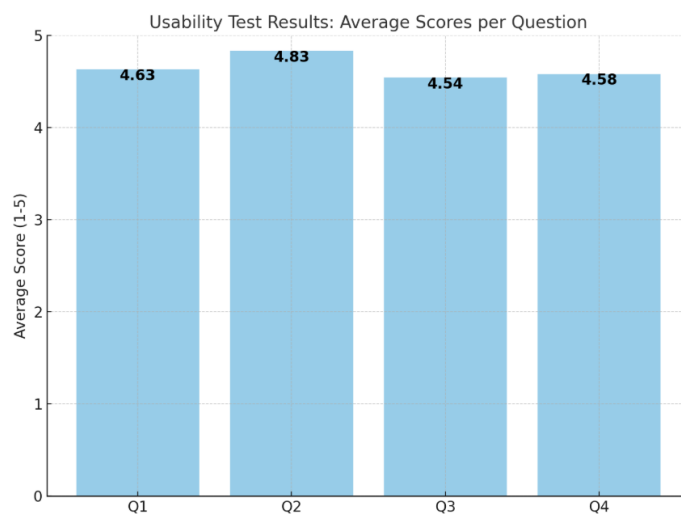


Figure 6.4: Usability questions’ response average.

## 7 CONCLUSION

This project aimed to prototype an intelligent EV charging slot recommendation and reservation system that addresses common limitations of today’s EV charging networks, particularly the inefficiencies of first-come, first-served models and simple reservation systems that do not anticipate future station load. While the problem context is motivated by the rapidly growing EV ecosystem in Turkey, the system was developed and tested using the Codalab Smart Mobility Data Challenge dataset [1], which provides rich historical station usage data suitable for training and testing predictive models. Since the training dataset spans from July 2020 to February 2021, covering several months but not a full year of seasonality, the resulting prediction model performed robustly within the available data range.

The developed solution integrates a machine learning-based availability prediction model, which forecasts future station availability using historical usage patterns and contextual features. This predictive capability is combined with Google Maps-based navigation and an intelligent Slot Fit Score calculation that ranks candidate charging stations based on estimated time of arrival, predicted availability, and current reservation status. The system is implemented as a native iOS application, developed with Swift and SwiftUI following an MVVM architecture. It leverages Firebase Firestore for nearly real-time data storage and reservation management, and Firebase Authentication for secure user account handling. Key features of the application include dynamic station recommendations, availability-aware reservation validation and management, and map-based navigation to selected stations. The resulting prototype demonstrates that predictive, data-driven reservation systems can enhance the efficiency and user experience of EV charging networks.

Several enhancements can further improve the system, particularly the integration of On-Board Diagnostics (OBD2)-based data collection. OBD2 is a standardized interface in modern vehicles that allows external devices to connect—typically via Bluetooth—and access real-time data from the car’s internal sensors. Leveraging this technology, the application could automatically obtain key vehicle information such as battery level, health, and charge status without requiring manual input from the user. This not only increases convenience but also improves the accuracy and timeliness of charging slot recommendations. Beyond data automation, the system could be extended with personalized features such as user-specific charging histories, behavioral analytics through interactive charts, and tailored suggestions based on past usage. Seamless integration of secure payment options and QR code-based station identification would further streamline the end-to-end user experience by enabling quick, contactless reservations and station access. Collectively, these enhancements would elevate both the functionality and usability of the system, aligning it more closely with the expectations of a modern, intelligent EV charging platform.

## 8 Bibliography

### References

- [1] AI for the Future of Mobility Challenge Organizers. Smarter mobility data challenge for a greener future. [https://codalab.lisn.upsaclay.fr/competitions/7192#learn\\_the\\_details](https://codalab.lisn.upsaclay.fr/competitions/7192#learn_the_details), 2023. Accessed: 2025-06-09.
- [2] Anadolu Ajansı. Türkiye’de elektrikli otomobil sayısı bir yılda yüzde 130 arttı, May 2024. Accessed: 2025-06-11.
- [3] Enerji Ekonomisi. Şarj istasyonu sayısı 26 bini geçti, May 2024. Accessed: 2025-06-11.
- [4] Binod Vaidya and Hussein T Mouftah. Smart electric vehicle charging management for smart cities. *IET Smart Cities*, 2(1):4–13, 2020.
- [5] Ashwani Kumar, Ravinder Kumar, and Ashutosh Aggarwal. S2rc: A multi-objective route planning and charging slot reservation approach for electric vehicles considering state of traffic and charging station. *Journal of King Saud University-Computer and Information Sciences*, 34(5):2192–2206, 2022.
- [6] Yue Cao, Tong Wang, Omprakash Kaiwartya, Geyong Min, Naveed Ahmad, and Abdul Hanan Abdullah. An ev charging management system concerning drivers’ trip duration and mobility uncertainty. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 48(4):596–607, 2016.
- [7] Simone Orcioni, Luca Buccolini, Adrianna Ricci, and Massimo Conti. Electric vehicles charging reservation based on ocpp. In *2018 IEEE International Conference on Environment and Electrical Engineering and 2018 IEEE Industrial and Commercial Power Systems Europe (EEEIC/I&CPS Europe)*, pages 1–6. IEEE, 2018.
- [8] Robert Basmadjian, Benedikt Kirpes, Jan Mrkos, Marek Cuchý, and Sahar Rastegar. An interoperable reservation system for public electric vehicle charging stations: A case study in germany. In *Proceedings of the 1st ACM International Workshop on Technology Enablers and Innovative Applications for Smart Cities and Communities*, pages 22–29, 2019.
- [9] Mohd Saqib, Md Muzakkir Hussain, Mohammad Saad Alam, MM Sufyan Beg, and Amol Sawant. Smart electric vehicle charging through cloud monitoring and management. *Technology and Economics of Smart Grids and Sustainable Energy*, 2:1–10, 2017.

- [10] Radu Flocea, Andrei Hîncu, Andrei Robu, Stelian Senocico, Andrei Traciu, Baltariu Marian Remus, Maria Simona Răboacă, and Constantin Filote. Electric vehicle smart charging reservation algorithm. *Sensors*, 22(8):2834, 2022.
- [11] Narayana Eedara, NVS K Chaitanya Sela, Priyanka Veeranala, and Jayanth Pedakapu. Machine learning for intelligent ev charging slot booking during low-cost grid periods. In *2024 IEEE 9th International Conference for Convergence in Technology (I2CT)*, pages 1–5. IEEE, 2024.
- [12] Suresh Chavhan, Nikhil Dubey, Abhinav Lal, Dev Khetan, Deepak Gupta, Ashish Khanna, Joel JPC Rodrigues, and Plácido Rogerio Pinheiro. Next-generation smart electric vehicles cyber physical system for charging slots booking in charging stations. *IEEE Access*, 8:160145–160157, 2020.
- [13] Sheng Cao, Sixuan Dang, Xiaojiang Du, Mohsen Guizani, Xiaosong Zhang, and Xiaoming Huang. An electric vehicle charging reservation approach based on blockchain. In *GLOBECOM 2020-2020 IEEE Global Communications Conference*, pages 1–6. IEEE, 2020.
- [14] Joseph K Liu, Willy Susilo, Tsz Hon Yuen, Man Ho Au, Junbin Fang, Zoe L Jiang, and Jianying Zhou. Efficient privacy-preserving charging station reservation system for electric vehicles. *The Computer Journal*, 59(7):1040–1053, 2016.
- [15] Safa Hmdare, David J Brown, Yue Cao, Mohammad Aljaidi, Omprakash Kaiwartya, Rahul Yadav, Pratik Vyas, and Manish Jugran. Ev charging management and security for multi-charging stations environment. *IEEE Open Journal of Vehicular Technology*, 2024.
- [16] Julian Timpner and Lars Wolf. Design and evaluation of charging station scheduling strategies for electric vehicles. *IEEE Transactions on Intelligent Transportation Systems*, 15(2):579–588, 2013.
- [17] Weijia Zhang, Hao Liu, Fan Wang, Tong Xu, Haoran Xin, Dejing Dou, and Hui Xiong. Intelligent electric vehicle charging recommendation based on multi-agent reinforcement learning. In *Proceedings of the Web Conference 2021*, pages 1856–1867, 2021.
- [18] Shuohan Liu, Xu Xia, Yue Cao, Qiang Ni, Xu Zhang, and Lexi Xu. Reservation-based ev charging recommendation concerning charging urgency policy. *Sustainable Cities and Society*, 74:103150, 2021.
- [19] Abdelhameed Ibrahim, El-Sayed M El-Kenawy, Marwa Metwally Eid, Abdelaziz A Abdelhamid, M El-Said, Amal H Alharbi, Doaa Sami Khafaga, Wael A Awad, Rawya Yehia

- Rizk, Nadjem Bailek, et al. A recommendation system for electric vehicles users based on restricted boltzmann machine and waterwheel plant algorithms. *IEEE Access*, 11:145111–145136, 2023.
- [20] Pannee Suanpang and Pitchaya Jamjuntr. Optimizing electric vehicle charging recommendation in smart cities: A multi-agent reinforcement learning approach. *World Electric Vehicle Journal*, 15(2):67, 2024.
- [21] Mostafa M Shibl, Loay S Ismail, and Ahmed M Massoud. Electric vehicles charging management using deep reinforcement learning considering vehicle-to-grid operation and battery degradation. *Energy Reports*, 10:494–509, 2023.
- [22] Rahul Mehta, Dipti Srinivasan, Ashwin M Khambadkone, Jing Yang, and Anupam Trivedi. Smart charging strategies for optimal integration of plug-in electric vehicles within existing distribution system infrastructure. *IEEE Transactions on Smart Grid*, 9(1):299–312, 2016.
- [23] Alain Aoun, Mehdi Adda, Adrian Ilinca, Mazen Ghandour, and Hussein Ibrahim. Dynamic charging optimization algorithm for electric vehicles to mitigate grid power peaks. *World Electric Vehicle Journal*, 15(7), 2024.
- [24] Lei Yang, Xinbo Geng, Xiaohong Guan, and Lang Tong. Ev charging scheduling under demand charge: A block model predictive control approach. *IEEE Transactions on Automation Science and Engineering*, 21(2):2125–2138, 2023.
- [25] Grammarly Blog. Understanding linear regression. <https://contenthub-static.grammarly.com/blog/wp-content/uploads/2024/09/156443-61046105blogvisuals-Linear-regression1.png>, 2024. Accessed: 2025-06-09.
- [26] Medium Blog. Understanding non-linear regression. [https://miro.medium.com/v2/resize:fit:583/0\\*2D6gg0Ejv0iIASCG.png](https://miro.medium.com/v2/resize:fit:583/0*2D6gg0Ejv0iIASCG.png), 2024. Accessed: 2025-06-09.
- [27] Fritz AI. Regression model visualization. [https://fritz.ai/wp-content/uploads/2023/09/1jojTznH4H0X\\_8cGw\\_040DA-1024x576.png](https://fritz.ai/wp-content/uploads/2023/09/1jojTznH4H0X_8cGw_040DA-1024x576.png), 2023. Accessed: 2025-06-08.
- [28] Janosh Riebesell. Random forest diagram. <https://tikz.net/janosh/random-forest.png>, n.d. Accessed: 2025-06-08.
- [29] "Medium". Regression model types illustration. [https://miro.medium.com/v2/resize:fit:1400/1\\*OZPOQUKiaVmZOEMm\\_-8iYA.png](https://miro.medium.com/v2/resize:fit:1400/1*OZPOQUKiaVmZOEMm_-8iYA.png), n.d. Accessed: 2025-06-08.

- [30] IntelligentSystemsLab. Spatio-temporal electric vehicle charging dataset in paris (st-evcdp). <https://github.com/IntelligentSystemsLab/ST-EVCDP>, 2023. Accessed: 2025-06-09.